Università degli Studi di Milano
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
GIOVANNI DEGLI ANTONI



CORSO DI LAUREA TRIENNALE IN
CORSO DI LAUREA

STUDIO E TESTING DELL'ATTACCO CRIME A TLS

Relatore: Prof. Andrea Visconti

Elaborato Finale di:
Michela Francesca Firrera
Matr. Nr. 02022A

Anno Accademico 2025-2026

Alla mia 'Ernacchia', alla migliore amica che non ho scelto, ma alla migliore che potessi desiderare

A mamma e papà, la mia strada di mattoni dorati verso la Città di Smeraldo

"Oh oh..."
"Non dirmelo... stiamo andando verso un'altissima cascata"
"Già"
"Con massi appuntiti"
"È un classico"
"... e andiamo"

– Kuzco, Le follie dell'imperatore

"Sai cosa succede quando sbagli direzione?"
"Cosa succede?"
"Niente"
"Come niente?"
"Niente. Se sbagli direzione, ti giri, torni indietro e ci riprovi"
"Ma così avrò perso tempo"
"Perché? Ne hai poco?"

– Megi Bulla, Milena insegnami la felicità

Ringraziamenti

E dopo qualche annetto di impegno, posso anche io finalmente arrivare al traguardo di questo percorso. Non è stato facile, per nulla, ho subito all'inizio un impatto che per diverso tempo non riuscivo a spiegare nemmeno a me stessa: la me del passato che aveva trovato tutto semplice si è trovata una realtà talmente nuova da rimanerne completamente tramortita e riprendersi ha richiesto tempo. Tuttavia, riuscire ad arrivare qua oggi, nonostante tutti gli ostacoli, mi riempie di soddisfazione. Rifarei tutto da capo ora che so com'è andata? In realtà sì, perché se all'inizio pensavo di dover fare tutto da sola, grazie a queste difficoltà ho capito che un traguardo non si raggiunge mai davvero da soli. Sì, sono qui grazie ai miei sforzi, ma anche e soprattutto grazie al sostegno, all'affetto e alla presenza di chi mi ha camminato a fianco in tutto questo tempo. Ed è a ciascuna di queste persone che voglio dedicare le righe che seguono, per dirvi un grazie che viene dal cuore.

Un ringraziamento speciale va al mio relatore, il professor Visconti, per avermi guidato in questo percorso, per la costante disponibilità e gentilezza. Grazie, non solo per i consigli dati, ma anche per l'entusiasmo con cui abbiamo sempre lavorato e, soprattutto, per aver creduto in me in ogni momento, permettendomi di lavorare in un ambiente sereno e stimolante.

A mamma e papà, a voi va il grazie più grande. Grazie per esserci sempre stati e aver creduto in me, anche quando ero io a dubitarne, per non avermi mai fatto pesare il tempo in più che ho dovuto prendermi per arrivare al traguardo, per tutti i sacrifici che avete fatto e per tutto l'amore che mi avete dato, magari non ve lo dico spesso, ma vi voglio bene in una maniera smisurata. Se oggi stringo questa tesi tra le mani, è perché vi ho sempre avuti accanto a me. Questo traguardo è vostro, tanto quanto mio.

A Rosy, alla mia "piccola" grande sorella. Grazie per essere il mio porto sicuro, quella che, ogni volta che ti racconto qualcosa che non va, si infervora anche più di me e prende le mie battaglie come se fossero le sue, facendomi sempre sentire difesa e mai sola. Sei l'oasi in mezzo al deserto che riesce sempre a liberarmi dai miei pensieri più pesanti e la prima a ricordarmi quanto valgo quando io me ne dimentico. Anche questo traguardo, in fondo, ha la tua energia dentro.

Ad Andre, Cri, Daniil, Laura, Matte, Rongo, Sara e Vale, ai vecchi e ai nuovi amici. Grazie di esserci stati nel mio periodo più difficile, di avermi supportato e di avermi dato una carica senza la quale non sarei andata avanti. Grazie di tutte le risate, delle vostre risposte serie alle mie domande del giorno decisamente poco serie, degli aneddoti serali e delle belle giornate tra giochi e spensieratezza. Siete stati la mia ancora nei momenti no e la mia felicità in tutti gli altri.

A quattro di loro, però, devo dedicare un grazie un po' più speciale.

A Matteo, il mio alter ego maschile. Grazie per tutto l'aiuto che mi hai dato nell'ultimo periodo, per avermi difesa e supportata in ogni mio traguardo e per essere sempre riuscito a strapparmi una risata anche quando ero triste. Tu e Sara siete il baluardo dell'amore a cui anch'io aspiro, e non potrei essere più felice di vedervi insieme.

A Vale. Ci siamo incontrate un po' per caso - "Galeotta fu quella Coca-Cola" - ma da lì è nata un'amicizia straordinaria. Il tuo supporto, le frasi di incoraggiamento e gli abbracci che mi dai ogni volta sono carezze che mi rincuorano in ogni istante: riesci a capire sempre come sto e ad esserci per me anche quando non dico nulla. Sei una delle ragazze più in gamba che io conosca: so per certo che spaccherai nella vita e spero di essere sempre al tuo fianco a fare il tifo per te. A te va anche un augurio speciale: c'è una laurea in arrivo anche per te, bellissima, e io sarò lì a festeggiarti, come sempre!

A Laura. Ti abbiamo un po' "adottata" tramite Sara, ma sei subito diventata una presenza di cui ora non posso più fare a meno. La tua dolcezza e la tua gentilezza riempiono il mio cuore di gioia anche nelle giornate più difficili, sei una ragazza d'oro, non cambiare mai. Grazie per non farmi mai sentire sola e per tutto il tuo appoggio; il modo in cui riesci a farmi sentire speciale è un dono prezioso, davvero incredibile. E grazie di essere sempre nerd con me quando si tratta di videogiochi. Ehi! voi che state leggendo, lei ha creato un gioco da zero e ha spaccato! Pronta ad affrontare insieme il prossimo livello?

E l'ultimo grazie va a Sara. È cominciato tutto con un timido "Ciao" a Crittografia, sfumato subito dopo per la troppa timidezza nel risalutarci, ed è arrivato fino a qui, a oggi. Oggi che non ti ho solo al mio fianco per darmi forza, ma ti ho letteralmente accanto: questo giorno è tanto mio quanto tuo e non avrei voluto nessun'altra al mio fianco per laurearmi. Il supporto, le risate e gli scleri condivisi sono tra i ricordi più belli di questo percorso. Aver trovato una persona così simile e compatibile a me è uno dei regali più grandi che potessi ricevere. Grazie, semplicemente grazie di tutto: non ci sono abbastanza parole per descrivere quanto ti sia grata per ciò che hai fatto e fai per me.

A Greta, Sara, Giulia e Leyla, con cui ho avuto la fortuna di crescere e che mi accompagnano da più tempo. A voi va un grazie immenso per essermi sempre state accanto, per

le serate a parlare fino a tardi e a giocare in maniera sempre 'poco' competitiva. Le vostre risate hanno alleggerito questo percorso non facile, regalandomi preziosi momenti di conforto e rifugio.

Un grazie speciale va a Sara. Un'amicizia cresciuta un passo alla volta, ma diventata ormai fondamentale. Grazie per le nostre infinite maratone di serie, per le risate fino alle lacrime e per il supporto che non mi fai mancare. La tua schiettezza è una delle cose che apprezzo di più in assoluto: grazie per essere sempre sincera, vera e al mio fianco.

Uno dei grazie più grandi va a Greta: in ogni traguardo importante che ho vissuto c'è sempre stata una presenza costante nella mia vita, ed è la tua. Il supporto e il sostegno che ho ricevuto, e che ricevo tuttora, sono smisurati e non ti dirò mai abbastanza grazie per questo. In ogni momento di sconforto tu c'eri, pronta a darmi la forza di ripartire. Sei una forza della natura, non scordarlo mai. Grazie di essere una delle certezze della mia vita, per avermi ascoltato e per starmi sempre vicino. Sono passati dieci anni da quando sei entrata nella mia vita e il regalo più bello è sapere che questo è solo l'inizio di tutti i prossimi che condivideremo.

Un grazie particolare va anche a chi non ho nominato esplicitamente, ma che ha fatto parte di questo percorso che mi ha portata fino a qui. Se stai leggendo questa tesi significa che, anche solo in una piccola parte, mi hai reso la persona che sono oggi e mi hai supportato in questo cammino, perciò grazie della tua presenza e dei bei ricordi costruiti insieme.

E dopo tanta fatica, penso che per una volta l'ultimo grazie doveroso sia a me stessa. In questi anni ho costantemente lottato con quella parte di me che era pronta ad arrendersi e che non credeva che questo potesse essere il suo posto, non riuscendo a vedere una luce in fondo al tunnel. Tuttavia, una parte di me, all'inizio più debole, ma poi sempre più presente invece ci ha creduto fino alla fine, cercando di contrastare tutti quei pensieri negativi che mi riempivano la testa. È vero, non sarei qui se non mi fossi ritrovata circondata da persone che mi amano e che mi vogliono bene, ma in primis mi sono voluta bene io per poter insistere e fidarmi che ci sarei riuscita e così è successo ed è di una soddisfazione disarmante poter scrivere finalmente queste parole.

A volte è difficile riuscire a credere a qualcosa che sembra impossibile, un miraggio. Eppure, in fondo, la lezione più grande che mi porto via da questo viaggio è proprio questa: non bisogna mai smettere di darsi una possibilità, di fidarsi del proprio istinto e di inseguire la propria felicità, costi quel che costi. Perché alla fine si arriva sempre là dove si merita di essere, anche solo per il gusto di voltarsi indietro e sorridere a quel miraggio che ora è realtà.

E Casomai non vi rivedessi, buon pomeriggio, buonasera e buonanotte!

Indice

Ringraziamenti **i**

Indice **iv**

1 Introduzione **1**

2 Crittografia **3**
2.1 Crittografia Simmetrica 4
2.1.1 Cifrari a flusso 4
2.1.2 Cifrari a blocchi 6
2.2 Crittografia Asimmetrica 7
2.2.1 RSA 8
2.2.2 Diffie-Hellman 8
2.3 Integrità dei dati e autenticazione 9
2.3.1 Funzioni di Hash 9
2.3.2 Messagge Authentication Code (MAC) e HMAC 9
2.3.3 Firme digitali e certificati X.509 10
2.4 Crittografia Ibrida 10

3 TLS e CRIME **12**
3.1 TLS 12
3.1.1 Differenza SSL e TLS 13
3.1.2 Architettura interna TLS 15
3.1.3 Crittografia ibrida e cipher suite 16
3.1.4 Meccanismo di handshake 17
3.1.5 Compressione dati 19
3.2 CRIME 21
3.2.1 Prerequisti dell'attacco 21
3.2.2 Funzionamento dell'attacco 22
3.2.3 Strategie di mitigazione 23

4 Analisi tecnica dell'implementazione **25**
4.1 L'oracolo di compressione: la funzione `misura()` . . . 25
4.2 Il ciclo byte-by-byte: ricostruzione automatica del segreto . . . 26
4.3 La logica match/mismatch: identificazione del carattere corretto . . . 27
4.4 I tre ruoli del Man-in-the-Middle: Server, Client e Attaccante . . . 27
4.5 Il thread delle iniezioni: attacco in parallelo al traffico legittimo . . . 29
4.6 I socket: comunicazione tra processi indipendenti . . . 30

5 Test **32**
5.1 Ripetizione medesimo carattere . . . 32
5.2 Match / Mismatch progressivo . . . 34
5.3 Confronto sistematico MATCH vs MISMATCH . . . 35
5.4 Lunghezza del segreto e intensità del segnale . . . 36
5.5 Attacco byte-by-byte automatizzato . . . 37
5.6 Mappa del segnale su tutti i caratteri . . . 38
5.7 CRIME vs brute force . . . 39
5.8 Man-in-the-middle silenzioso . . . 41
5.8.1 Esempio HTTP . . . 43
5.8.2 Rilevamento anomalie . . . 44
5.8.3 Contromisura . . . 45
5.8.4 Processi Separati . . . 46
5.9 Valutazione globale del lavoro di testing . . . 48

6 Conclusioni e sviluppi futuri **50**

Bibliografia **52**

Capitolo 1

Introduzione

Navigare su Internet, effettuare un acquisto online o semplicemente accedere alla propria casella di posta elettronica sono azioni che oggi compiamo quotidianamente, spesso senza soffermarci su quanto accade dietro le quinte. Ogni singola interazione in rete si fonda su uno scambio continuo di dati sensibili che viaggiano attraverso nodi e canali intrinsecamente insicuri. Garantire che questi dati arrivino a destinazione in modo sicuro e senza essere alterati è fondamentale per permettere a chiunque di usare il web in tranquillità.

In questo scenario, la crittografia rappresenta il fulcro della sicurezza informatica moderna, fornendo le basi teoriche per la creazione di canali di comunicazione protetti. Tra le varie soluzioni sviluppate nel tempo, il protocollo TLS si è imposto come lo standard di riferimento per cifrare il traffico web.

Tuttavia, un sistema non è mai sicuro solo perché gli algoritmi che lo compongono sono matematicamente robusti. Spesso, le vulnerabilità nascono proprio dall'interazione inattesa tra funzionalità diverse. È esattamente questo il caso dell'attacco CRIME, una minaccia che nel 2012 ha scosso la comunità scientifica dimostrando come due meccanismi pensati per migliorare l'efficienza e la sicurezza, ovvero la compressione dei dati e la cifratura del traffico, potessero trasformarsi, se combinati, in un canale di attacco. Sfruttando la semplice variazione di lunghezza dei pacchetti cifrati, un osservatore esterno è in grado di risalire, carattere dopo carattere, a informazioni estremamente riservate.

Questo lavoro di tesi nasce con l'obiettivo di analizzare da vicino questo confine tra efficienza e sicurezza, esplorando l'evoluzione dei protocolli trasmissivi e approfondendo il funzionamento pratico dell'attacco CRIME.

Il percorso si articola attraverso quattro capitoli principali, con il primo ci concentreremo sui fondamenti della crittografia, fornendo i concetti di riservatezza, integrità e autenticazione, e analizzando le differenze strutturali tra cifratura simmetrica e asimmetrica;

successivamente, il focus si sposterà sul protocollo TLS, ripercorrendone l'evoluzione da SSL fino a TLS 1.3. In questa sezione esamineremo l'architettura interna del protocollo, il meccanismo di handshake e il ruolo svolto dalla compressione dei dati; a questo punto si analizzerà l'attacco CRIME: ne studieremo i prerequisiti di rete, la logica dell'estrazione dei segreti e le risposte che hanno portato alla completa rimozione della compressione nelle specifiche moderne del protocollo; infine, l'elaborato si concluderà con la parte pratica dello studio, incentrata sulla realizzazione di una simulazione dell'attacco in un ambiente controllato. Attraverso lo sviluppo di script dedicati, mostreremo concretamente le dinamiche dell'iniezione dei dati, le modalità di rilevamento della minaccia e le contromisure necessarie per garantire la protezione del canale.

Capitolo 2

Crittografia

La sicurezza delle comunicazioni sulle reti informatiche si basa su un requisito fondamentale ovvero la capacità di proteggere le informazioni sensibili da possibili intercettazioni o manipolazioni da parte di soggetti non autorizzati. Storicamente nata come l'arte di nascondere il significato dei messaggi, la crittografia moderna si è evoluta in una disciplina scientifica e matematica più complessa. Oggi essa non rappresenta soltanto uno strumento per cifrare, e rendere quindi illeggibili i dati, ma costituisce anche la struttura abilitante per qualsiasi protocollo di comunicazione sicuro, primo fra tutti il protocollo TLS, spiegato in dettaglio nel Capitolo 3.

Per poter comprendere a pieno i meccanismi di protezione che governano il traffico web, è doveroso definire i tre pilastri essenziali della sicurezza dell'informazione [1] che la crittografia si propone di garantire:

- Riservatezza (Confidenzialità): Assicura che l'informazione sia accessibile e comprensibile esclusivamente ai legittimi destinatari. Attraverso algoritmi di cifratura, i dati in chiaro vengono trasformati in un testo cifrato che risulta indistinguibile da rumore casuale per chiunque non sia in possesso della chiave di decifrazione;
- Integrità: Garantisce che i dati non vengano alterati, modificati o corrotti durante il transito, sia a causa di errori fisici di trasmissione sia a causa di un intervento doloso da terze parti. La crittografia fornisce strumenti matematici per rilevare all'istante anche la minima variazione di un singolo bit nel payload;
- Disponibilità: Assicura che gli utenti autorizzati possano accedere ai dati e alle risorse di rete ogni volta che ne hanno la necessità.
- Autorizzazione: Determina e vincola i diritti di accesso alle risorse, garantendo che un utente autenticato possa compiere esclusivamente le azioni per le quali gli sono

stati concessi i privilegi.

- Autenticazione: Permette di verificare con certezza l'identità dichiarata da un utente coinvolto nella comunicazione.

Mentre la disponibilità viene solitamente garantita da misure infrastrutturali, di ridondanza e di bilanciamento del carico, la crittografia si concentra principalmente sulla risoluzione matematica dei problemi lagati alla riservatezza e all'integrità. A queste proprietà si affianca un quarto pilastro cruciale per le comunicazioni di rete [2]: l'autenticazione, ovvero la capacità di verificare con certezza l'identità dei soggetti coinvolti prima di avviare lo scambio di dati.

I moderni protocolli di sicurezza combinano questi obiettivi attraverso l'uso di chiavi crittografiche. A seconda di come queste chiavi vengono distribuite e utilizzate dai partecipanti alla sessione, la crittografia si divide in due grandi categorie, cioè la crittografia simmetrica (o a chiave privata) e la crittografia asimmetrica (o a chiave pubblica). Nei paragrafi seguenti verranno analizzati il funzionamento, i limiti e le applicazioni di entrambe le tecnologia, ponendo le basi per comprendere come esse si fondano all'interno del protocollo TLS.

2.1 Crittografia Simmetrica

La crittografia simmetrica [3], conosciuta anche come crittografia a chiave privata o condivisa, rappresenta la forma più antica e intuitiva di cifratura. Il suo principio di funzionamento è lineare, ovvero i soggetti coinvolti nella comunicazione utilizzano la medesima chiave segreta sia per cifrare il messaggio originale (testo in chiaro) sia per decifrare il messaggio ricevuto (testo cifrato), come mostrato in figura 1.

Questo approccio offre prestazioni computazionali elevate, rendendo gli algoritmi simmetrici ideali per la protezione di grandi volumi di dati in transito o memorizzati su disco. Tuttavia, presenta una limite intrinseco essenziale, vale a dire la distribuzione della chiave. Prima di poter avviare una sessione sicura, le parti devono concordare e scambiarsi la chiave segreta attraverso un canale sicuro, un problema complesso in reti pubbliche e aperte come Internet.

In base al modo in cui elaborano le informazioni in ingresso, gli algoritmi di cifratura simmetrica si dividono in due grandi categorie, ossia i cifrari a flusso e i cifrari a blocchi.

2.1.1 Cifrari a flusso

I cifrari a flusso elaborano il testo in chiaro un bit alla volta [4]. Il cuore di questo meccanismo è un generatore di numeri pseudocasuali che, partendo dalla chiave segreta, produce

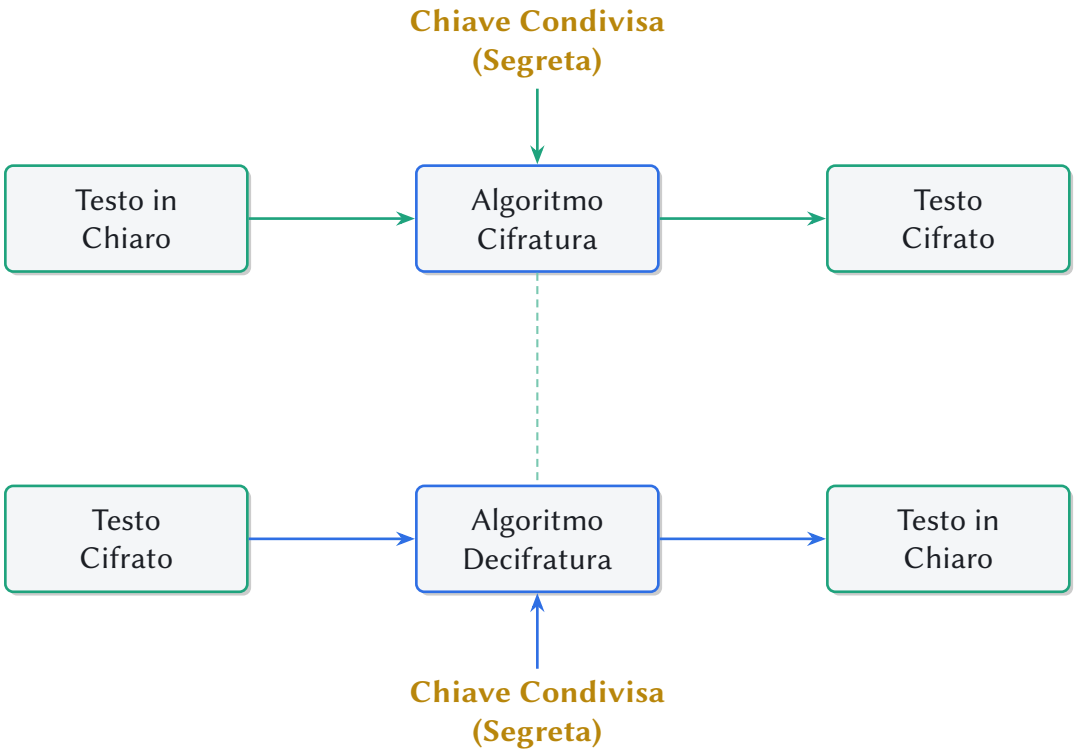


Figura 1: Cifratura Simmetrica

una sequenza continua di bit priva di pattern ripetitivi, definita keystream. La cifratura avviene combinando direttamente, tramite l'operatore logico XOR, ciascun bit del messaggio con il corrispondete bit del keystream:

$$C_i = P_i \oplus K_i$$

Dove C_i rappresenta il bit cifrato, P_i il bit in chiaro e K_i il bit del keystream. La decifratura segue lo stesso principio logico, andando a sfruttare la proprietà di reversibilità dello XOR.

In questo ambito, l'esempio più rilevante è RC4, un algoritmo molto veloce e leggero che in passato ha protetto buona parte del traffico web all'interno delle prime versioni di SSL e TLS. Pur essendo efficienti, i cifrari a flusso sono altamente sensibili a errori di implementazione: se la stessa porzione di keystream viene riutilizzata per cifrare due messaggi diversi con la stessa chiave, un attaccante può facilmente ricostruire i testi in chiaro originari. Proprio a causa di queste fragilità strutturali e di vulnerabilità emerse nel tempo, l'uso di RC4 e dei cifrari a flusso tradizionali è stato progressivamente deprecato e infine rimosso dagli standard di sicurezza moderni.

2.1.2 Cifrari a blocchi

A differenza dei cifrari a flusso, i cifrari a blocchi non elaborano i dati in modo continuo, ma suddividono il testo in chiaro in porzioni di dimensione fissa, chiamate blocchi (solitamente da 64 o 128 bit) [5]. L'algoritmo elabora ciascun blocco in modo indipendente o concatenato, applicando una serie complessa di sostituzioni e permutazioni regolate dalla chiave segreta. L'adozione della cifratura a blocchi introduce due requisiti architetturali fondamentali:

Padding

Poiché la dimensione del messaggio originale raramente coincide con un multiplo esatto della dimensione del blocco, si rende necessatio applicare una tecnica di riempimento definita padding. Il protocollo aggiunge in coda alll'ultimo blocco una sequenza di byte fittizi per raggiungere la lunghezza minima richiesta dall'algoritmo prima di avviare la cifratura. Sebbene il padding consenta di uniformare i dati, la sua gestione introduce sottili perdite di informazioni a livello logico, che hanno storicamente costretto gli sviluppatori a rivedere la gestione degli errori nelle varie versioni di TLS.

Modalità di Funzionamento

Se un blocco di testo viene cifrato sempre con la stessa chiave, input identici produranno output identici, consentendo a un ossservatore esterno di individuare pattern ripetitivi nel traffico. Per evitare questa vulnerabilità, si utilizzano diverse modalità di funzionamento che collegano matematicamente i blocchi tra loro:

- Cipher Block Chaining (CBC): In questa modalità, ciascun blocco di testo in chiaro viene sottoposto a un'operazione di XOR con il blocco cifrato immediatamente precedente prima di essere a sua volta cifrato. Per iniziare il processo con il primo blocco, viene utilizzato un Vettore di Inizializzazione (IV), una stringa di byte casuali che garantisce che messaggi identici cifrati con la stessa chiave producano testi cifrati completamente diversi. La corretta gestione e imprevedibilità dell'IV è un fattore critico; debolezze nella sua implementazione hanno esposto il protocollo TLS 1.0 e 1.1 a severi attacchi;
- Galois / Counter Mode (GCM): Rappresenta lo standard moderno di riferimento. È una modalità di cifratura autenticata che non si limita a garantire la riservatezza del dato, ma calcola contestualmente un tag di autenticazione, eliminando alla radice la necessità del padding tradizionale e proteggendo la sessione da attacchi di manipolazione attiva del traffico.

2.2 Crittografia Asimmetrica

La crittografia asimmetrica, o a chiave pubblica, nasce negli anni '70 per risolvere il più grande limite strutturale della crittografia simmetrica: la necessità di scambiare una chiave segreta su un canale insicuro senza che nessuno possa intercettarla.

A differenza dei sistemi simmetrici, gli algoritmi asimmetrici utilizzano una coppia di chiavi distinte ma matematicamente correlate tra loro [6]:

- Chiave Pubblica: È liberamente accessibile e distribuibile a chiunque. Viene utilizzata dal mittente per cifrare il messaggio o per verificare una firma digitale;
- Chiave Privata: Deve rimanere segreta e sotto l'esclusivo controllo del legittimo proprietario. Viene utilizzata per decifrare i messaggi cifrati con la corrispondente chiave pubblica o per apporre una firma digitale.

Il principio matematico sottostante si basa su funzioni unidirezionali dette *Trapdoor one–way functions* ovvero operazioni matematiche facili da eseguire in un senso, ma praticamente impossibili da invertire senza conoscere l'informazione segreta (la chiave privata).

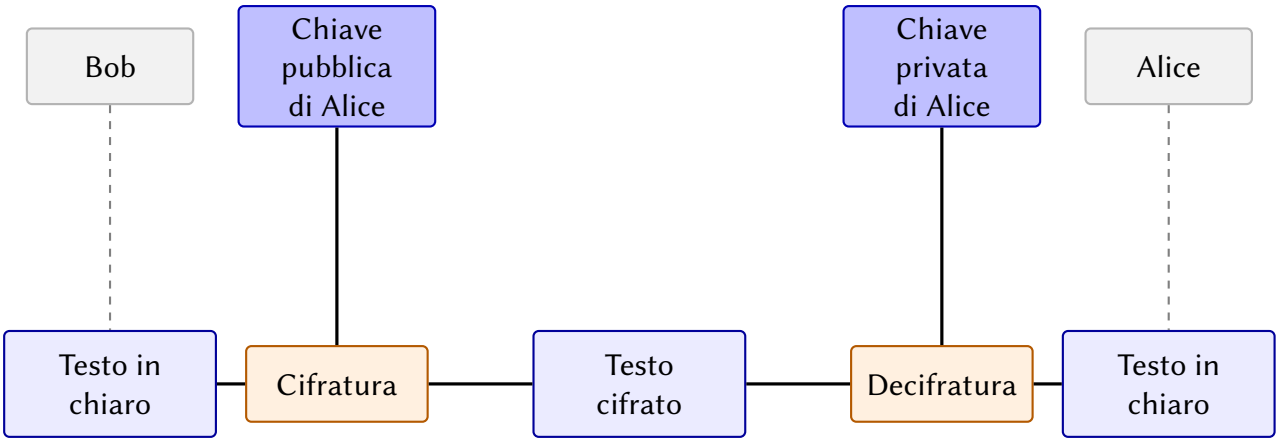


Figura 2: Cifratura Asimmetrica

Come mostrato nella Figura 2, il flusso di comunicazione si articola in passaggi ben definiti:

- Il mittente acquisisce la chiave pubblica del destinatario e la utilizza per applicare l'algoritmo di cifratura al messaggio in chiaro;
- Il messaggio viene trasformato in un testo cifrato, che può viaggiare in sicurezza sul canale di comunicazione pubblico;

- Ricevuto il pacchetto, il destinatario applica l'algoritmo di decifratura sfruttando la propria chiave privata, ripristinando il messaggio originale.

Questo approccio elimina il problema della distribuzione iniziale delle chiavi, ossia un client può connettersi a un server sconosciuto, prelevarne la chiave pubblica e stabilire un canale cifrato. Tuttavia, la complessa matematica sottostante rende la crittografia asimmetrica sensibilmente più lenta rispetto a quella simmetrica, rendendola inadatta alla cifratura di grandi volumi di dati applicativi.

I due algoritmi di riferimento, utilizzati da TLS; sfruttano principi matematici differenti:

2.2.1 RSA

RSA basa la sua sicurezza sulla difficoltà computazionale di fattorizzare numeri interi molto grandi prodotti dalla moltiplicazione di due numeri primi segreti. Nonostante, sia semplice da comprendere, RSA presenta un'architettura rigida: la stessa coppia di chiavi può essere utilizzata sia per l'autenticazione sia per lo scambio di una chiave segreta [7].

Nell'handshake delle prime versioni di TLS, il client generava la chiave simmetrica finale, la cifrava con la chiave pubblica RSA del server e gliela inviava. Sa da un lato questo approccio semplificava il processo, dall'altro introduceva un grave rischio per la sicurezza a lungo termine legato alla mancanza di FOrwsrd Secrecy.

2.2.2 Diffie-Hellman

L'algoritmo Diffie-Hellman (DH) non nasce per cifrare i dati, ma specificatamente come protocollo di accordo sulla chiave. Basandosi sulla difficoltà del calcolo del logaritmo discreto, permette a due controparti di concordare un segreto comune scambiandosi solamente parametri pubblici su un canale insicuro, senza che la chiave finale viaggi mai in rete [8].

La sua evoluzione si basa sulle curve ellittiche (ECDH), che garantisce lo stesso livello di sicurezza matematica di RSA ma con chiavi molto più corte, riducendo l'impatto sulla CPU e la dimensione dei pacchetti di rete.

L'uso di Diffie-Hellman in modalità effimera, in cui le chiavi assimmetriche cambiano ad ogni singola sessione e vengono distrutte subito dopo l'uso, abilita la proprietà di Perfect Forward Secrecy.

Questa proprietà garantisce che, anche se un attaccante dovesse registrare oggi tutto il traffico cifrato di un sito web e, successivamente, riuscisse a rubare la chiave privata del

server, non sarebbe comunque in grado di decifrare le vecchie comunicazioni memorizzate. Con RSA la compromissione della chiave del server comprometteva l'intero storico del traficco; con Diffie-Hellman ogni sessione è protetta in modo indipendente.

In TLS, questo porta alla completa rimozione del supporto alla scambio chiave tramite RSA a favore dell'uso esclusivo di varianti effimere di Diffie-Hellman.

2.3 Integrità dei dati e autenticazione

La sola cifratura del messaggio non è sufficiente a garantire una comunicazione sicura. Un attaccante posizionato sul canale di rete potrebbe infatti modificare i bit del testo cifrato in transito, cancellarne delle parti o sostituire l'intero pacchetto senza che il destinatario possa accorgersene al momento della decifratura. Per questo motivo, la crittografia possiede strumenti specifici dedicati alla verifica dell'integrità e all'autenticazione delle fonti.

2.3.1 Funzioni di Hash

Una funzione crittografica di hash è un algoritmo matematico deterministico che prende in input un messaggio di lunghezza arbitraria e lo trasforma in una stringa di lunghezza fissa, chiamata digest o impronta digitale [9].

Le funzioni di hash devono garantire tre proprietà fondamentali:

- Resistenza alla preimmagine: Dato un digest, deve essere computazionalmente impossibile risalire al messaggio originale;
- Resistenza alla preimmagine secondaria: Dato un messaggio, deve essere impossibile trovarne un secondo diverso che generi lo stesso identico digest;
- Resistenza alle collisioni: Deve essere estremamente difficile individuare due messaggi differenti che producano il mdesimo digest.

Gli algoritmi più conosciuti appartengono alla famiglia Secure Hash Algorithm (SHA). All'interno dei protocolli di rete, l'hash viene utilizzato per sintetizzare i dati, ma da solo non protegge da attacchi nativi. Se un attaccante modifica il messaggio, può semplicemente ricalcolare l'hash del nuovo messaggio e inviarlo al destinatario.

2.3.2 Messagge Authentication Code (MAC) e HMAC

Per legare l'integrità all'autencità del mittente, si utilizza MAC. Il MAC combina il messaggio in chiaro con una chiave segreta condivisa tra le parti prima di generare l'impronta.

La variante più comune è HMAC che applica la funzione di hash in due passaggi successivi incorporando la chiave segreta. Soltanto chi possiede la chiave segreta può generare o verificare un HMAC valido: se un attaccante modifica il pacchetto durante il trasporto, non riuscirà a ricalcolare l'HMAC corretto, e il destinatario scarterà immediatamente il messaggio manomesso.

Nel TLS Record Protocol, prima dell'avvento della cifratura autenticata AEAD, ogni pacchetto applicativo veniva sottoposto al calcolo di un HMAC prima o dopo la cifratura per garantire che i dati non venissero alterati lungo la rotta TCP.

2.3.3 Firme digitali e certificati X.509

Mentre il MAC richiede una chiave segreta già condivisa, la firma digitale sfrutta la crittografia asimmetrica per garantire l'autenticità verso terzi. Per firmare un documento:

- Il mittente calcola l'hash del messaggio;
- Cifra l'hash ottenuto utilizzando la propria chiave privata;
- Chiunque riceva il messaggio può decifrare la firma usando la chiave pubblica del mittente e confrontare l'hash ottenuto con quello calcolato sul messaggio originale.

Tuttavia, il client non può essere certo che una determinata chiave pubblica sia davvero appartenente al server legittimo e non di un possibile impostore. A questo scopo nascono i Certificati Digitali X.509 e l'infrastruttura Public Key Infrastructure (PKI). Un certificato digitale è un documento elettronico che associa in modo vincolante l'identità di un soggetto alla sua chiave pubblica. Tale associazione viene garantita e firmata digitalmente da un'entità terza di fiducia riconosciuta a livello globale: la Certificate Authority (CA).

2.4 Crittografia Ibrida

Come analizzato nei paragrafi precedenti, né la crittografia simmetrica né quella asimmetrica rappresentano da sole la soluzione completa per la protezione dei flussi di rete reali:

- La crittografia simmetrica è estremamente veloce ed efficiente nel cifrare grandi moli di dati, ma richiede uno scambio sicuro della chiave che non può avvenire direttamente su canali pubblici;
- La crittografia asimmetrica risolve il problema della distribuzione delle chiavi e dell'autenticazione tramite certificati, ma comporta un carico computazionale eccessivo per la gestione del traffico continuo.

I moderni protocolli di comunicazione sicura risolvono questo dilemma adottando una struttura a crittografia ibrida, un approccio combinato che sfrutta i punti di forza di entrambe le tecnologie [10]:

- Fase di negoziazione (Asimmetrica): Inizialmente le parti utilizzano la crittografia asimmetrica per autenticarsi a vicenda e stabilire in modo sicuro una chiave segreta temporanea, detta chiave di sessione.
- Fase di trasferimento dati (Simmetrica): Una volta concordata la chiave di sessione, la crittografia asimmetrica viene messa da parte. Il traffico applicativo vero e proprio viene cifrato e protetto in modo rapido tramite cifratura simmetrica.

Questo modello costituisce il motore fondamentale del protoccollo TLS, la cui architettura e il cui funzionamento dettagliato verranno esaminati nel capitolo successivo.

Capitolo 3

TLS e CRIME

In questo Capitolo viene presentata una panoramica approfondita sul protocollo TLS, fulcro del presente studio, ripercorrendone l'evoluzione storica dalla nascita fino alle versioni attuali e delineandone il funzionamento logico. Successivamente, l'analisi si focalizza sull'attacco CRIME, esaminandone dettagliatamente il meccanismo d'azione e le specifiche vulnerabilità del protocollo che ne permettono lo sfruttamento.

3.1 TLS

Il protocollo TLS (Transport Layer Security) [11] è uno standard crittografico ampiamente diffuso, progettato per garantire la riservatezza, l'integrità dei dati e l'autenticazione all'interno di reti TCP/IP, come Internet. La necessità di preservare la riservatezza dei dati rappresenta tuttora un requisito fondamentale, in particolare durante il transito di informazioni sensibili quali credenziali di accesso, transazioni bancarie o dettagli delle carte di credito. In assenza di protezione, tali flussi di rete risultano vulnerabili a intercettazioni da parte di malintenzionati, con il conseguente rischio di esfiltrazione o manipolazione dei dati stessi. Per ovviare a queste minacce, si rende indispensabile l'adozione di tecnologie crittografiche capaci di cifrare il traffico e assicurarne l'integrità durante il trasporto. Proprio per questo, TLS fornisce le seguenti proprietà:

- Autenticazione: assicura che l'identità delle controparti coinvolte nella comunicazione sia verificata e certificata. Un utente attaccante potrebbe impersonare l'identità di un interlocutore e ricevere informazioni riservate, perciò è necessario certificare l'identità delle parti remote;
- Integrità: garantisce che i dati trasmessi non subiscano modifiche lungo il percorso. Il protocollo permette di rilevare qualsiasi alterazione del payload, sia essa causata

da errori accidentali di trasmissione o da interventi malevoli di terze parti mediante attacchi di manipolazione del traffico;

- Riservatezza: impone che i dati siano accessibili e interpretabili esclusivamente dai partecipanti autorizzati alla sessione. Attraverso l'uso della crittografia, i flussi informatici vengono resi completamente intelligibili a chiunque cerchi di intercettarli sul canale di comunicazione.

Il protocollo TLS opera idealmente al di sopra del livello di trasporto e immediatamente al di sotto del livello applicativo, fungendo da strato intermedio di sicurezza. Questa specifica collocazione architetturale gli consente di intercettare i dati generati dall'applicazione, cifrarli e veicolarli in modo sicuro verso il sottostante protocollo TCP. Al contempo, l'indipendenza dal livello superiore permette a TLS di offrire i propri servizi di protezione a una vasta gamma di protocolli applicativi. Il caso d'uso più celebre è rappresentato dall'estensione di HTTP, ma lo standard è impiegato per porre in sicurezza altri servizi fondamentali, quali SMTP, FTP, IMAP e POP3.

3.1.1 Differenza SSL e TLS

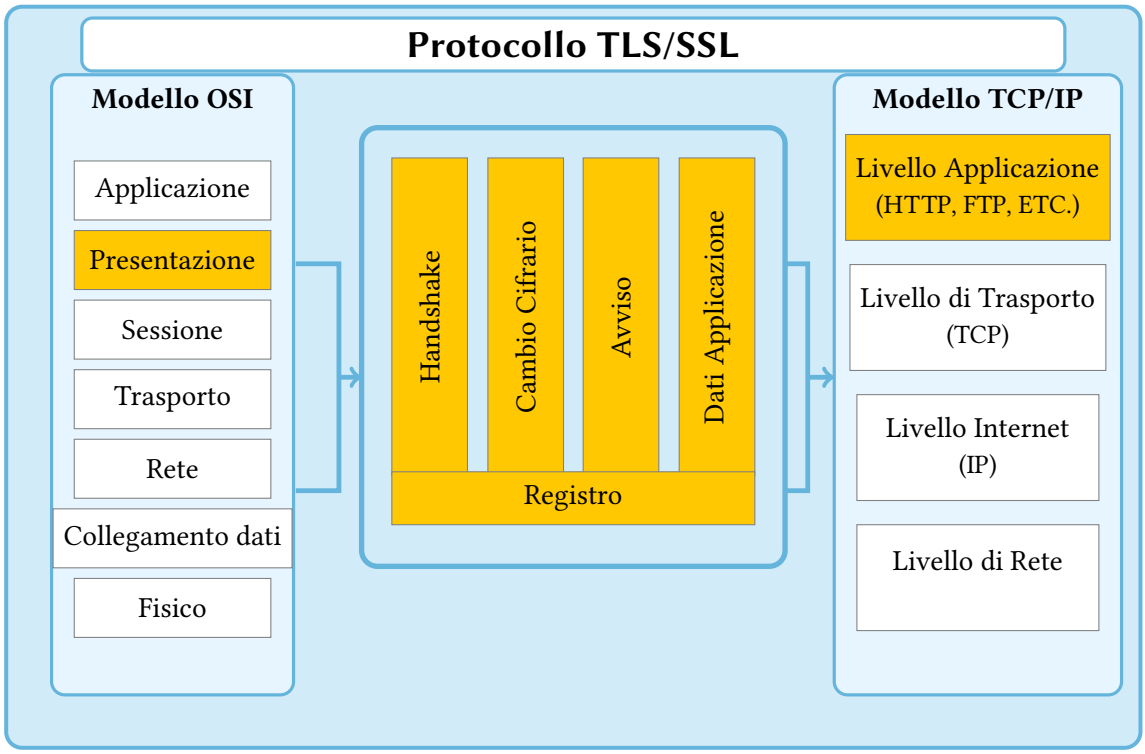


Figura 3: Protocollo TLS su modello OSI

Il protocollo TLS è l'evoluzione diretta del precedente standard di cifratura SSL (Secure Sockets Layer) [12], originariamente sviluppato da Netscape. La sua prima versione, SSL 1.0, non fu mai rilasciata pubblicamente a causa di gravi e strutturali vulnerabilità di sicurezza. Di conseguenza, il debutto ufficiale avvenne nel 1995 con il rilascio di SSL 2.0. Nonostante ciò, anche questa versione si rivelò ben presto affetta da criticità significative, spingendo gli sviluppatori a sostituirla l'anno successivo, nel 1996, con SSL 3.0. Quest'ultima release rappresentò una profonda riscrittura del protocollo, ma non rimase esente da difetti stabili nel tempo; in seguito alla scoperta di nuove e severe falle di sicurezza, l'IETF (Internet Engineering Task Force), l'organismo responsabile della definizione degli standard tecnici di Internet, ne decretò il definitivo deprezzamento.

Nel 1999, nasce quindi la prima versione di TLS, TLS 1.0 [13] che migliora in quattro aspetti in confronto a SSL 3.0. In primo luogo, l'architettura è stata ripensata per garantire una solida sicurezza crittografica, prerequisito indispensabile per l'instaurazione di canali di comunicazione protetti tra due controparti remote. A questa si affianca la necessità di un'interoperabilità universale, concepita per consentire agli sviluppatori di implementare soluzioni TLS in grado di negoziare e scambiare parametri crittografici in modo standardizzato.

Il protocollo è stato, poi, strutturato secondo criteri di estensibilità, offrendo un framework flessibile e modulare capace di accogliere e integrare, qualora si rendesse necessario in futuro, nuovi algoritmi a chiave pubblica. Infine, i progettisti hanno prestato particolare attenzione all'efficienza relativa delle operazioni. Poiché i computi crittografici, specialmente quelli asimmetrici, comportano un elevato carico di lavoro per la CPU, TLS 1.0 ha introdotto un meccanismo opzionale di memorizzazione in cache delle sessioni, concepito specificamente per ridurre il numero di handshake completi da eseguire da zero e ottimizzare così le prestazioni complessive della rete.

La prima versione di TLS rimane in vigore per sette anni, fino al rilascio di TLS 1.1 nel 2006, una versione che, però, non apporta grandi modifiche alla precedente e che, perciò, viene sostituita solamente due anni dopo dalla 1.2. In particolare, ha migliorato la gestione dei vettori di inizializzazione e il sistema di notifiche degli errori per arginare gli attacchi contro la modalità di cifratura CBC, aggiungendo inoltre le prime descrizioni per evitare nuove minacce informatiche emerse fino a quel momento.

I cambiamenti di maggiore impatto arrivano, tuttavia, con la versione 1.2 [14] che innalza notevolmente il livello di protezione, garantisce agli sviluppatori una flessibilità maggiore nella scelta e nella gestione dei pacchetti di cifratura e fornisce nuove descrizioni per difendersi dai primi attacchi che TLS inizia a subire quali l'attacco di Biechenbacher.

Nel 2018, infine, esce l'ultima versione di TLS, TLS 1.3 [15] che introduce rinnovamenti architetturali volti a ottimizzare sia la sicurezza che le prestazioni. Tra le novità principali

spicca l'implementazione di un meccanismo di protezione contro gli attacchi di downgrade, progettato per impedire a un utente malintenzionato di forzare la comunicazione verso versioni più vecchie e vulnerabili del protocollo.

Inoltre, modifica le modalità di negoziazione iniziale, introducendo un sistema più robusto e flessibile per la scelta della versione da utilizzare. Sul fronte della validazione e della crittografia asimmetrica, vengono adottati schemi di firma digitale decisamente più sicuri e moderni, offrendo al contempo al client la capacità di comunicare preventivamente al server quali specifici algoritmi sia in grado di verificare all'interno dei certificati di sicurezza. Il risultato è un protocollo più snello, che elimina i vecchi algoritmi obsoleti e velocizza sensibilmente l'instaurazione delle connessioni sicure.

3.1.2 Architettura interna TLS

Come evidenziato nello schema architetturale illustrato in Figura 3, il protocollo TLS non va concepito come un blocco monolitico, bensì come una struttura stratificata suddivisa in due macro-livelli operativi, come spiegato in [16]. Il livello superiore è composto da quattro sotto-protocolli specifici, ognuno deputato a una precisa funzione di controllo o di trasferimento dati, mentre il livello inferiore è interamente occupato dal TLS Record Protocol, il quale funge da strato di astrazione e trasporto per i moduli soprastanti.

I quattro moduli che governano le operazioni logiche della sessione TLS interagiscono costantemente per garantire l'allineamento tra client e server:

- Handshake Protocol: Rappresenta la componente più complessa e cruciale dell'intera architettura. Interviene nella fase iniziale della connessione, consentendo alle parti remote di autenticarsi reciprocamente, negoziare la versione dello standard da utilizzare, concordare gli algoritmi crittografici (mediante la scelta della cipher suite come verrà spiegato nella Sezione 3.1.3) e stabilire in modo sicuro il materiale segreto da cui verranno derivate le chiavi di sessione;
- Change Cipher Spec Protocol: È il sotto-protocollo più snello, costituito da un unico messaggio di un solo byte. La sua funzione è esclusivamente segnaletica: notifica alla controparte che, da quel preciso istante in poi, tutti i record successivi saranno protetti e cifrati secondo le chiavi concordate durante l'handshake;
- Alert Protocol: Gestisce la segnalazione delle anomalie e la messaggistica di notifica. I messaggi si dividono in due livelli di gravità: *warning*, che segnalano condizioni insolite senza interrompere la sessione, e *fatal*, che determinano l'immediato abbattimento della connessione e la distruzione delle chiavi associate;
- Application Data Protocol: È il modulo responsabile del trasporto effettivo dei dati generati dai livelli applicativi superiori. Una volta che la sessione sicura è stata

stabilita con successo, i flussi informativi di protocolli come HTTP, FTP o SMTP vengono veicolati all'interno di questo canale, beneficiando in modo trasparente delle proprietà di riservatezza e integrità offerte dall'infrastruttura sottostante.

Tutti i messaggi prodotti dai quattro sotto-protocolli descritti vengono incapsulati e processati dal TLS Record Protocol. Questo strato inferiore si comporta come un motore operativo che riceve i dati dai moduli superiori e applica loro una serie di trasformazioni sequenziali prima di affidarli al livello di trasporto TCP.

Il processo di elaborazione segue un flusso logico ben definito. In primo luogo, i flussi di dati in ingresso vengono frammentati in blocchi di dimensioni gestibili. Su ciascun frammento viene applicata, se prevista dalla versione del protocollo in uso e dalla negoziazione, la compressione dei dati. Successivamente il Record Protocol calcola un codice di controllo per l'integrità, il *Message Authentication Code* (MAC), e procede alla cifratura simmetrica dell'intero pacchetto. Infine viene aggiunto un header standard contenente i metadati essenziali prima che il pacchetto venga inoltrato lungo la pila TCP/IP verso la destinazione.

3.1.3 Crittografia ibrida e cipher suite

Per svolgere i propri obiettivi di sicurezza TLS adotta un approccio basato sulla crittografia ibrida, che combina i punti di forza delle due principali famiglie crittografiche: la crittografia asimmetrica e quella simmetrica.

La crittografia asimmetrica, basata su coppie di chiavi matematicamente correlate come in RSA o Diffie-Hellman, risolve il problema fondamentale della distribuzione delle chiavi su canali insicuri e permette alle parti di autenticarsi reciprocamente. Tuttavia, la sua elevata complessità matematica la rende computazionalmente costosa e inadatta alla protezione del traffico dati di massa.

Per questo motivo, una volta completata la fase di handshake e concordato un segreto comune, TLS abbandona la crittografia asimmetrica e passa alla crittografia simmetrica. Algoritmi come AES utilizzano un'unica chiave condivisa per cifrare e decifrare i dati: sono veloci ed efficienti, risultando ideali per la protezione del flusso continuo di dati applicativi.

Struttura di una cipher suite

L'insieme specifico di algoritmi crittografici utilizzati per proteggere una determinata sessione prende il nome di Cipher Suite. Durante la prima fase della connessione, il client invia un elenco delle suite da lui supportate e il server seleziona quella che ritiene più idonea tra quelle compatibili con le proprie configurazioni.

Nello standard TLS 1.2, una tipica cipher suite viene identificata da una stringa standardizzata che descrive analiticamente le quattro componenti fondamentali del meccanismo di protezione adottato. Con l'avvento di TLS 1.3 nel 2018, l'architettura delle cipher suite è stata semplificata. L'IETF ha rimosso il supporto a tutti gli algoritmi considerati obsoleti o deboli e ha separato i meccanismi di autenticazione e scambio delle chiavi dalla suite stessa. Di conseguenza, in TLS 1.3 le suite contengono soltanto l'algoritmo di cifratura simmetrica e la funzione di hash. Questa riduzione ha diminuito la superficie d'attacco del protocollo, eliminando configurazioni potenzialmente insicure, e ha velocizzato la fase di negoziazione iniziale, poiché il ventaglio di opzioni da esaminare è stato ridotto a pochi standard di massima sicurezza.

3.1.4 Meccanismo di handshake

Il protocollo di handshake [17] è la procedura iniziale attraverso la quale client e server stabiliscono i parametri di sicurezza prima di avviare qualsiasi scambio di dati applicativi. Gli obiettivi principali di questa fase sono l'autenticazione delle controparti, la scelta della cipher suite e la generazione delle chiavi simmetriche di sessione. L'evoluzione del protocollo ha portato a una ristrutturazione di questo flusso tra le versioni TLS 1.2 e TLS 1.3, orientata alla riduzione della latenza e all'eliminazione di passaggi ridondanti.

Nello standard TLS 1.2, il processo di handshake completo richiede due tempi di andata e ritorno dei pacchetti (2-RTT) come mostrato in Figura 4. Il flusso logico si articola in una serie di messaggi sequenziali:

- *ClientHello*: il client avvia la connessione inviando un messaggio che contiene la versione massima di TLS supportata, una stringa di byte casuali per il calcolo delle chiavi e l'elenco delle cipher suites e dei metodi di compressione disponibili;
- *ServerHello*: il server risponde selezionando la versione del protocollo e la singola cipher suite che verranno utilizzate, inviando a sua volta una propria stringa casuale;
- *Server Certificate*: il server invia il proprio certificato digitale firmato da una Certificate Authority (CA) fidata, permettendo al client di verificarne l'identità radiografando la chiave pubblica in esso contenuta;
- *Server Key Exchange (Opzionale)*: se la suite scelta lo richiede, il server invia i parametri matematici necessari per lo scambio delle chiavi. Infine, invia il messaggio di ServerHello Done per indicare la fine della sua fase di avvio;
- *Client Key Exchange*: il client verifica la validità del certificato del server. Successivamente, genera un ulteriore segreto, lo cifra con la chiave pubblica del server (o calcola la sua quota tramite Diffie-Hellman) e lo trasmette al server;

- *Generazione della Master Secret*: a questo punto, sia il client che il server possiedono il Client Random, il Server Random e il Pre-Master Secret. Attraverso una funzione pseudo-casuale (PRF), entrambe le parti calcolano in modo indipendente la medesima Master Secret, dalla quale verranno derivate le chiavi simmetriche per la cifratura e l'integrità;
- *Change Cipher Spec e Finished*: entrambi i nodi si scambiano un messaggio per notificare l'attivazione della cifratura, seguito dal messaggio di Finished, che contiene un riepilogo cifrato di tutti i messaggi passati per verificare che l'handshake non sia stato alterato da un attaccante Man-in-the-Middle.

Uno dei traguardi più significativi di TLS 1.3 è la riduzione del tempo di handshake a un singolo RTT, spiegato in dettaglio in [18], dimezzando i tempi di attesa rispetto alla versione precedente. Questa ottimizzazione è resa possibile da un approccio denominato *Key Share*. Poiché TLS 1.3 supporta soltanto un ristretto insieme di algoritmi di scambio delle chiavi basati su curve ellittiche, il client non si limita a inviare l'elenco delle cipher suite nel *ClientHello*: ipotizza gli algoritmi che il server potrebbe preferire e invia contestualmente la propria porzione di chiave pubblica per ciascuno di essi.

Il server, ricevuto il *ClientHello*, sceglie la cipher suite, calcola il segreto condiviso e risponde con il proprio *ServerHello* comprensivo di *Key Share*, certificato e messaggio *Finished*. Al termine di questo primo scambio entrambe le parti dispongono già delle chiavi crittografiche necessarie e possono avviare immediatamente la trasmissione dei dati applicativi cifrati.

TLS 1.3 introduce, inoltre, la modalità *0-RTT* per la ripresa della sessione: se client e server si sono già connessi in precedenza, il client può utilizzare i parametri crittografici memorizzati dalla sessione precedente per inviare dati cifrati già nel primo messaggio, azzerando completamente la latenza di negoziazione.

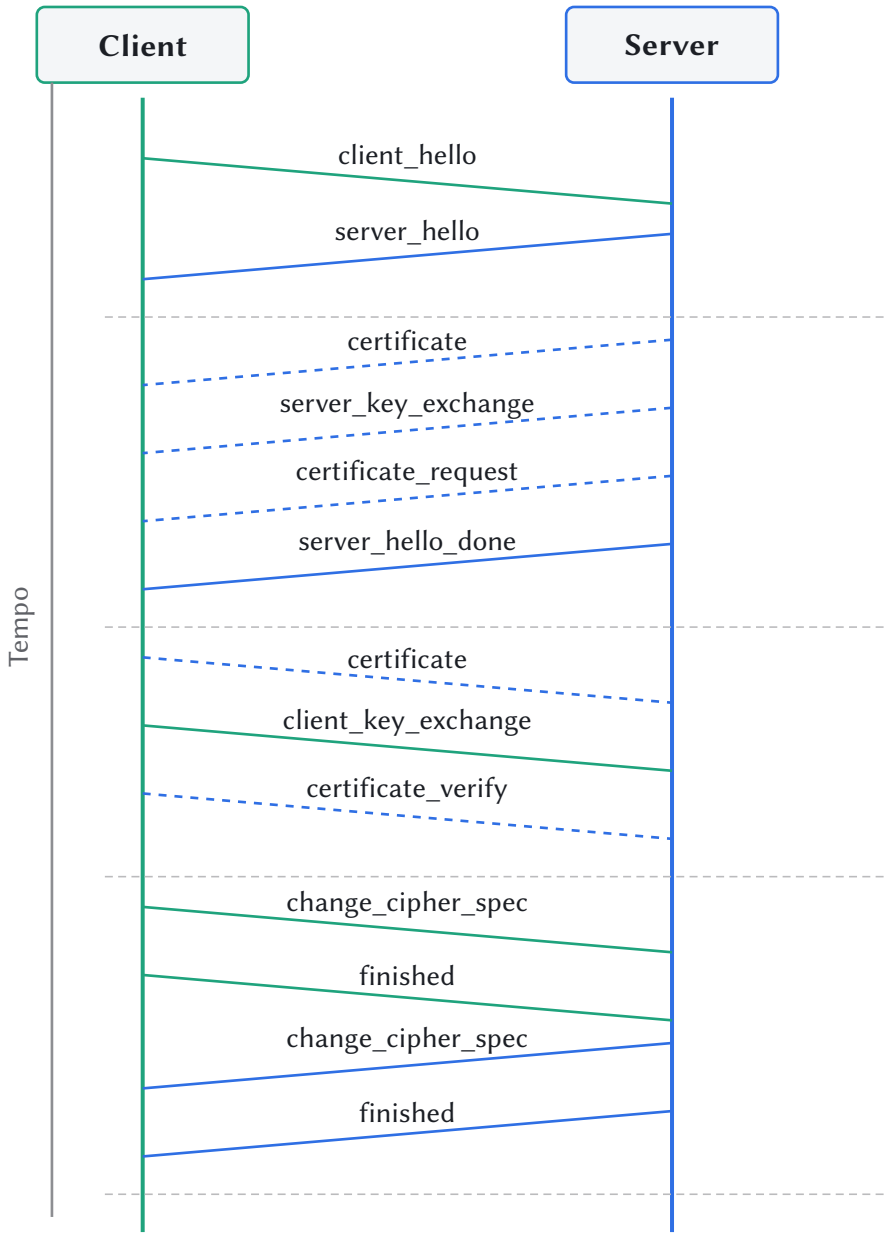


Figura 4: Protocollo di handshake

3.1.5 Compressione dati

Nelle prime specifiche del protocollo TLS, fino alla versione 1.2, i progettisti hanno incluso il supporto nativo alla compressione dei dati a livello di Record Protocol. L'obiettivo

primario di questa funzionalità era l'ottimizzazione delle prestazioni di rete: comprimendo il payload applicativo prima di sottoporlo alla cifratura, si riduceva la quantità di byte da trasmettere sul canale, con un sensibile risparmio di banda e tempi di caricamento più rapidi, fattori cruciali soprattutto nelle reti a bassa connettività dell'epoca.

Funzionamento della compressione (algoritmo DEFLATE)

Lo standard di compressione maggiormente impiegato in TLS era l'algoritmo DEFLATE, che combina la codifica LZ77 e la codifica di Huffman, come illustrato in dettaglio in [19]. Il principio fondamentale su cui si basa è l'eliminazione della ridondanza informativa attraverso l'individuazione di sequenze di caratteri ripetute all'interno di un flusso di dati.

Quando DEFLATE [20] analizza un testo, ad esempio una richiesta HTTP, opera secondo una logica lineare: ogni volta che incontra una sequenza di caratteri già apparsa in precedenza all'interno del medesimo blocco di dati, la sostituisce con un puntatore alla sua prima occorrenza, composto da distanza e lunghezza. Ne consegue che maggiore è il numero di ripetizioni presenti nel payload, minore sarà la dimensione in byte del pacchetto risultante. Al contrario, flussi di dati casuali o privi di pattern ricorrenti non beneficeranno di alcuna riduzione significativa.

Il TLS Record Protocol esegue queste operazioni secondo una sequenza rigida: i dati applicativi vengono prima compressi e soltanto successivamente cifrati e trasmessi. Sebbene questo ordine sia logico dal punto di vista dell'efficienza, cifrare dati già compressi richiede meno calcoli, mentre comprimere dati già cifrati è impossibile poiché la crittografia li rende indistinguibili da rumore casuale privo di ridondanze, esso introduce un pericoloso effetto collaterale.

La cifratura ha il compito di nascondere il contenuto del messaggio, trasformandolo in una stringa di byte apparentemente casuale. Tuttavia non nasconde la lunghezza del messaggio originale: la dimensione del pacchetto cifrato finale è direttamente proporzionale alla dimensione del payload dopo la compressione. Si viene così a creare una correlazione pericolosa tra il grado di ridondanza dei dati in chiaro e la lunghezza del pacchetto che viaggia in rete. Questa sottile perdita di informazioni, apparentemente innocua, rappresenta la vulnerabilità teorica fondamentale che permette a un attaccante di dedurre informazioni riservate senza dover scardinare matematicamente gli algoritmi crittografici.

Questo specifico difetto architetturale ha aperto la strada a una classe di minacce del tutto nuova, il cui esponente più celebre nel contesto del traffico web è l'attacco CRIME, analizzato approfonditamente nel paragrafo seguente.

3.2 CRIME

Presentato ufficialmente nel settembre 2012 dai ricercatori di sicurezza Juliano Rizzo e Thai Duong durante la conferenza Ekoparty, l'attacco CRIME [21] (Compression Ratio Info-leak Made Easy) ha rappresentato una svolta nel campo della sicurezza applicata ai protocolli di trasporto sicuro. Fino a quel momento, la maggioranza degli sforzi della comunità scientifica era focalizzata sulla ricerca di debolezze negli algoritmi di cifratura o sulla scoperta di falle logiche nell'implementazione del software. CRIME, al contrario, ha dimostrato come sia possibile compromettere la riservatezza di una sessione HTTPS sfruttando un canale laterale (side-channel attack) generato dall'interazione di due funzionalità di per sé considerate sicure: la compressione dei dati e la cifratura.

L'obiettivo primario di questo attacco non è la forzatura matematica delle chiavi crittografiche o la rottura di cifrari robusti come AES, operazioni che richiederebbero risorse computazionali fuori dalla portata odierna. Il reale bersaglio di CRIME è l'estrazione mirata dei token di autenticazione e, nello specifico, dei cookie di sessione HTTPS.

Nelle moderne applicazioni web, i cookie vengono utilizzati come carta d'identità digitale dell'utente: una volta effettuato il login su un portale, il server rilascia un cookie contenente una stringa segreta e univoca. Da quel momento in poi, il browser della vittima allegherà automaticamente tale cookie a ogni successiva richiesta verso quel server per dimostrare l'avvenuta autenticazione, evitando di dover reinserire le credenziali ad ogni click.

Riuscire a intercettare e decifrare questo valore segreto permette a un utente malintenzionato di compiere un attacco di furto di sessione. L'attaccante può clonare il cookie all'interno del proprio browser e clonare l'identità della vittima sul sito target, ottenendo il pieno controllo del suo account senza mai essere entrato in possesso della password legittima. CRIME ha dimostrato empiricamente che, se la compressione TLS è attiva, un utente malintenzionato può ricostruire questo cookie segreto un carattere alla volta, analizzando unicamente le dimensioni dei pacchetti cifrati scambiati in rete.

3.2.1 Prerequisti dell'attacco

Per comprendere la logica dell'attacco CRIME, è importante delineare lo scenario operativo in cui si muove l'attaccante e i prerequisiti tecnici necessari alla sua esecuzione. L'attacco non si consuma in modo passivo, ma richiede un'interazione dinamica tra l'attaccante, la vittima e il server bersaglio. Le condizioni necessarie sono tre:

- Abilitazione della compressione: Sia il client (il browser della vittima) sia il server web devono supportare e negoziare attivamente un algoritmo di compressione a livello di trasporto o di protocollo applicativo evoluto. Gli standard interessati sono

tipicamente l'algoritmo DEFLATE o, in alternativa, il protocollo SPDY (precursore di HTTP/2), all'epoca largamente impiegato per velocizzare il traffico web;

- Posizione di Man-in-the-Middle (MitM): L'attaccante deve trovarsi in una posizione di rete che gli consenta di intercettare e monitorare il traffico della vittima. Questo posizionamento non serve a decifrare direttamente i dati, ma a misurare con precisione chirurgica la lunghezza in byte dei pacchetti cifrati che transitano sul canale;
- Iniezione di codice nel client: Il malintenzionato deve disporre di un vettore per eseguire script arbitrari nel contesto del browser della vittima. Lo scenario tipico vede l'utente navigare su un sito web maligno o compromesso tenendo contemporaneamente aperta, in un'altra scheda, la sessione sul sito bersaglio. Attraverso l'uso di script dedicati, l'attaccante può forzare il browser a emettere richieste HTTP ripetute e asincrone verso il sito target.

3.2.2 Funzionamento dell'attacco

Il fulcro di CRIME risiede nella capacità dell'attaccante di manipolare l'input della richiesta HTTP in modo da indurre l'algoritmo di compressione a rivelare informazioni sulla parte segreta del messaggio, sfruttando la ridondanza dei dati. Quando il browser effettua una richiesta HTTP forzata dallo script, l'header conterrà due elementi cruciali viaggiare nello stesso blocco:

- Il cookie segreto inserito automaticamente dal browser;
- Un parametro arbitrario inserito dall'attaccante tramite lo script.

L'algoritmo DEFLATE, operando prima della cifratura, analizza l'intero payload alla ricerca di stringhe duplicate da sostituire con puntatori alla prima occorrenza. L'attaccante sfrutta questo comportamento eseguendo una tecnica di forza bruta mirata, carattere per carattere, osservando la variazione della lunghezza complessiva del pacchetto cifrato finale:

- Fase di tentativo: Sapendo che i cookie di sessione utilizzano generalmente caratteri esadecimali o alfanumerici, lo script invia una serie di richieste in cui tenta di indovinare il primo carattere del cookie. L'attaccante inietta nel campo controllato la stringa Cookie: secret=0, poi Cookie: secret=1, e così via.
- Analisi della compressione: Se il carattere iniettato dall'attaccante è errato, la stringa non trova corrispondenze nel resto del testo. L'algoritmo non può comprimerla e la lunghezza del pacchetto finale rimane invariata. Se invece l'attaccante inietta il carattere corretto (ad esempio il 7), la stringa Cookie: secret=7 coinciderà parzialmente con l'inizio del vero cookie della vittima.

- La perdita di informazioni (Info-leak): Trovando una ridondanza, DEFLATE sostituisce il testo duplicato con un puntatore più corto. Di conseguenza, la lunghezza complessiva della richiesta HTTP compressa diminuisce di un byte. Poiché la cifratura preserva la dimensione del payload, l'attaccante in ascolto sulla rete vedrà il pacchetto cifrato rimpicciolirsi.

Una volta confermata la riduzione della dimensione del pacchetto, l'attaccante sa di aver indovinato il primo carattere. Lo script aggiorna immediatamente la stringa di ricerca fissando il valore indovinato (es. Cookie: secret=7) e inizia a testare i caratteri successivi (70, 71, 72 ... 7a). Ripetendo questo processo iterativo a cascata, il malintenzionato è in grado di ricostruire l'intero token di sessione in pochissimo tempo e con un numero estremamente ridotto di tentativi complessivi, completando il furto dell'identità digitale della vittima.

3.2.3 Strategie di mitigazione

La scoperta di CRIME ha scosso profondamente il settore della sicurezza informatica, poiché è riuscito ad attaccare un dogma consolidato: l'idea che la compressione e la cifratura potessero essere sovrapposte senza particolari cautele. L'impatto dell'attacco è stato immediato, al punto da costringere i principali produttori di software e gli organismi di standardizzazione a correre al riparo con contromisure drastiche.

Poiché la vulnerabilità risiede nella natura stessa dell'algoritmo di compressione applicato a dati parzialmente controllati dall'utente, l'unica mitigazione realmente efficace e strutturale si è rivelata essere la disattivazione totale della compressione nei contesti sensibili, mostrata in [22].

- Intervento sui browser e server web: Subito dopo la conferenza del 2012, i team di sviluppo dei principali browser hanno rilasciato aggiornamenti per disabilitare il supporto alla compressione DEFLATE all'interno del protocollo TLS. Parallelamente, i produttori di server web hanno modificato le loro configurazioni di default per rifiutare la negoziazione della compressione a livello di trasporto.
- L'abbandono di SPDY: Il protocollo SPDY, anch'esso vulnerabile a causa del suo meccanismo di compressione degli header, è stato progressivamente modificato e infine deprecato, spianando la strada alla nascita diHTTP/2, il quale adotta sistemi di compressione radicalmente diversi e specificamente progettati per non prestarsi a questo genere di attacchi side-channel.
- La standardizzazione in TLS 1.3: La risposta definitiva a livello architetturale è arrivata con la stesura di TLS 1.3 nel 2018. All'interno della specifica ufficiale l'IETF ha completamente eliminato il campo relativo alla compressione dal messaggio di

ClientHello, proibendo di fatto qualsiasi tentativo di compressione a livello di Record e sradicando la vulnerabilità alla radice.

In conclusione, CRIME ha ridefinito i criteri di progettazione dei sistemi sicuri, dimostrando alla comunità ingegneristica che la protezione del dato non può prescindere dall'analisi dei metadati, come la lunghezza dei pacchetti, e che l'efficienza sistemistica (il risparmio di banda tramite compressione) non deve mai essere ottenuta a scapito della robustezza della sicurezza globale.

Capitolo 4

Analisi tecnica dell'implementazione

Nel presente capitolo vengono illustrati i componenti algoritmici fondamentali alla base della simulazione dell'attacco CRIME. Non verrà mostrato l'intero codice sorgente, ma i soli frammenti rilevanti per la comprensione del meccanismo di attacco, evidenziando per ciascuno il ruolo che ha svolto nella riuscita degli esperimenti.

4.1 L'oracolo di compressione: la funzione `misura()`

Il nucleo dell'intera simulazione è una funzione denominata `misura()`, presente in tutti gli esperimenti eseguiti. Essa costituisce l'oracolo di compressione ovvero il componente che, dato un segreto e un payload, restituisce la dimensione in byte del dato compresso, replicando il comportamento del TLS Record Protocol.

```
1 def misura(secret, payload):
2     return len(zlib.compress(secret + payload, 6))
```

Listing 4.1: Funzione misura()

Questa funzione incapsula il principio teorico su cui si basa l'intero attacco CRIME. Il parametro `secret` rappresenta il dato segreto già presente nel dizionario del compressore, come ad esempio il cookie di sessione della vittima, mentre `payload` è la stringa inviata dall'attaccante nel tentativo di indovinare il segreto. I due vengono concatenati e compressi insieme tramite la libreria `zlib` al livello di compressione 6, corrispondente al livello predefinito di OpenSSL.

Il valore restituito, ovvero la lunghezza in byte del risultato compresso, è l'unica informazione che l'attaccante osserva in una connessione TLS reale. Se il payload contiene caratteri già presenti nel dizionario del compressore, ZLIB li sostituisce con puntatori interni, riducendo la dimensione del risultato. È proprio questa riduzione, spesso di un solo

byte, che rivela la corrispondenza tra il payload e il segreto, senza che il contenuto cifrato venga mai decifrato.

4.2 Il ciclo byte-by-byte: ricostruzione automatica del segreto

Il ciclo byte-by-byte è l'approccio che trasforma l'oracolo di compressione in un attacco vero e proprio. Utilizzato nell'attacco byte-by-byte automatizzato, nel Man-in-the-Middle silenzioso e nella simulazione HTTP, permette di ricostruire automaticamente un segreto completo senza conoscerlo a priori, operando un carattere alla volta.

```
for pos in range(len(secret_suffix)):
    known_so_far = KNOWN_PREFIX + discovered
    context      = SECRET + SECRET + SECRET
    min_size     = 99999
    best_char    = "?"

    for char in CHARSET:
        size = misura(context, known_so_far + char)
        if size < min_size:
            min_size  = size
            best_char = char

    discovered += best_char
```

Listing 4.2: Ciclo byte-by-byte

Il ciclo esterno itera su ogni posizione del segreto da scoprire. Per ciascuna posizione, il ciclo interno prova metodicamente tutti i caratteri dell'alfabeto `CHARSET` e misura la dimensione del pacchetto compresso per ognuno. Il carattere che produce la dimensione minima, ovvero quello che il compressore riconosce come già presente nel dizionario, viene selezionato come il carattere corretto e aggiunto alla variabile `discovered`.

Ci sono due aspetti da notare in questa implementazione. Il primo riguarda la costruzione del contesto: la variabile `context` viene ottenuta ripetendo tre volte il segreto noto, arricchendo il dizionario del compressore e rendendo il segnale più affidabile. Il secondo riguarda la variabile `known_so_far`, che accumula progressivamente i caratteri già scoperti: ogni nuovo tentativo parte da una base più ricca, consentendo al compressore di trovare corrispondenze sempre più lunghe e producendo un segnale sempre più preciso.

4.3 La logica match/mismatch: identificazione del carattere corretto

La logica match/mismatch è il ragionamento che sta alla base della distinzione tra un tentativo corretto e uno errato. Utilizzata esplicitamente nel match/mismatch progressivo, nel confronto sistematico e nella mappa del segnale, costituisce il fondamento teorico su cui si basa il ciclo byte-by-byte descritto nella sezione precedente.

```
sizes = {}
for char in CHARSET:
    sizes[char] = misura(SECRET, PREFIX + char)

min_size  = min(sizes.values())
best_char = min(sizes, key=sizes.get)
```

Listing 4.3: Logica match/mismatch

Per ogni carattere dell'alfabeto viene misurata la dimensione del pacchetto compresso ottenuto concatenando il prefisso noto con il carattere tentato. I risultati vengono raccolti nel dizionario `sizes`, che associa a ciascun carattere la dimensione del pacchetto corrispondente.

Il carattere corretto viene identificato come quello che produce la dimensione minima: `best_char` è il carattere associato al valore più piccolo in `sizes`. Questa scelta si basa direttamente sul principio dell'oracolo di compressione: il compressore ZLIB, trovando una corrispondenza tra il carattere tentato e il contenuto già presente nel dizionario, produce un pacchetto più compatto rispetto a tutti i caratteri sbagliati, che non beneficiano di alcuna ottimizzazione.

Va sottolineato che tutti i caratteri errati producono esattamente la stessa dimensione di pacchetto, indipendentemente da quale carattere sbagliato venga utilizzato. Questa proprietà rende il segnale binario e affidabile: non vi è ambiguità tra i mismatch e il carattere corretto si distingue in modo netto e univoco dal resto dell'alfabeto.

4.4 I tre ruoli del Man-in-the-Middle: Server, Client e Attaccante

La simulazione del Man-in-the-Middle silenzioso, utilizzata nella simulazione base e nella simulazione HTTP, è strutturata attorno a tre classi distinte che modellano i ruoli dei tre attori coinvolti nella comunicazione. Ciascuna classe opera in modo indipendente e non ha visibilità diretta sulle operazioni delle altre.

```
class Server:
    def __init__(self, secret):
        self.secret = secret

    def comprimi_e_invia(self, messaggio):
        dato = self.secret + messaggio
        return zlib.compress(dato, 6)


class Client:
    def invia(self, server, messaggio):
        return server.comprimi_e_invia(messaggio)


class Attaccante:
    def __init__(self, server, prefisso_noto):
        self.server        = server
        self.prefisso_noto = prefisso_noto
        self.scoperto      = ""

    def inietta(self, carattere):
        tentativo = self.prefisso_noto + self.scoperto +
    carattere
        context   = self.server.secret + self.server.secret +
    self.server.secret
        dato      = context + tentativo
        return len(zlib.compress(dato, 6))
```

Listing 4.4: Le tre classi del MitM

La classe `Server` modella il comportamento del server TLS vulnerabile: riceve un messaggio, lo concatena con il segreto in memoria e lo comprime prima di restituirlo. Questa operazione replica fedelmente il comportamento del TLS Record Protocol, in cui la compressione avviene prima della cifratura e coinvolge sia i dati applicativi sia il contesto già presente nel dizionario del compressore.

La classe `Client` è minimale: si limita a inviare messaggi al server e a riceverne le risposte, senza alcuna consapevolezza della presenza dell'attaccante. Questa semplicità riflette la condizione reale della vittima, che comunica normalmente senza accorgersi di essere intercettata.

La classe `Attaccante` è il componente più articolato. Il metodo `inietta()` costruisce un payload composto dal prefisso noto, dalla parte di segreto già scoperta e dal carattere tentato, lo comprime insieme al contesto arricchito e restituisce la dimensione del risultato. È

importante notare che l'attaccante non interagisce mai direttamente con il client: osserva il traffico e inietta i propri payload in modo completamente trasparente, senza alterare la comunicazione legittima.

4.5 Il thread delle iniezioni: attacco in parallelo al traffico legittimo

Nell'esperimento con processi separati, l'attaccante deve operare in parallelo al traffico legittimo del client senza interromperlo. Per realizzare questo comportamento è stato introdotto un thread dedicato alle iniezioni, che gira in modo indipendente rispetto al thread principale che gestisce il traffico del client.

```
def thread_iniezioni():
    secret_suffix = SECRET[len(KNOWN_PREFIX):]
    scoperto      = []

    for pos in range(len(secret_suffix)):
        min_size  = 99999
        best_char = "?"

        for char in CHARSET:
            tentativo  = KNOWN_PREFIX + "".join(scoperto) + char
            context    = SECRET + SECRET + SECRET
            compressed = zlib.compress(context + tentativo, 6)
            dim        = len(compressed)
            if dim < min_size:
                min_size  = dim
                best_char = char

        scoperto.append(best_char)

t_iniezioni = threading.Thread(target=thread_iniezioni)
t_iniezioni.start()
```

Listing 4.5: Thread delle iniezioni

Il thread viene creato tramite il modulo `threading` di Python e avviato con `t_iniezioni.start()`, che lo lancia in esecuzione concorrente rispetto al thread principale. Da quel momento i due thread operano in parallelo: il thread principale continua a ricevere le richieste del client e a inoltrarle al server, mentre il thread delle iniezioni esegue autonomamente il ciclo byte-by-byte per ricostruire il segreto.

Questo approccio dimostra concretamente la natura silenziosa dell'attacco CRIME: le iniezioni avvengono in modo completamente indipendente dal traffico legittimo, senza che il client o il server possano accorgersi della loro presenza.

4.6 I socket: comunicazione tra processi indipendenti

L'esperimento con processi separati rappresenta l'evoluzione più significativa dell'intera simulazione. A differenza degli esperimenti precedenti, in cui i tre attori coesistevano all'interno dello stesso script, in questo caso ogni processo comunica con gli altri tramite socket TCP, replicando il modello di comunicazione di una rete reale.

```
# Lato Server: apertura del socket in ascolto
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((SERVER_HOST, SERVER_PORT))
server_socket.listen(5)

conn, addr = server_socket.accept()
dati       = conn.recv(4096)
compressed = zlib.compress(SECRET + dati, 6)
dim        = str(len(compressed))
conn.send(dim)
conn.close()

# Lato MitM: intercettazione e inoltro
mitm_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
mitm_socket.bind((SERVER_HOST, MITM_PORT))
mitm_socket.listen(5)

conn, addr = mitm_socket.accept()
dati       = conn.recv(4096)

server_conn = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_conn.connect((SERVER_HOST, SERVER_PORT))
server_conn.send(dati)
risposta = server_conn.recv(1024)
server_conn.close()

conn.send(risposta)
conn.close()
```

Listing 4.6: Inizializzazione e gestione dei socket

Il server apre un socket TCP in ascolto sulla porta `SERVER_PORT` e attende connessioni in entrata. Alla ricezione di una richiesta, la concatena con il segreto, la comprime e restituisce al mittente la dimensione del risultato compresso, replicando il comportamento del TLS Record Protocol.

Il MitM apre a sua volta un socket in ascolto sulla porta `MITM_PORT`, su cui il client si connette invece di connettersi direttamente al server. Alla ricezione di una richiesta dal client, il MitM apre una nuova connessione verso il server reale, gli inoltra la richiesta, riceve la risposta e la restituisce al client senza modificarla. Dal punto di vista del client, la comunicazione avviene normalmente: non vi è alcuna differenza osservabile rispetto a una connessione diretta al server.

Questa architettura a tre socket, ovvero client verso MitM, MitM verso server, server verso MitM, replica la topologia di un attacco Man-in-the-Middle su rete, in cui l'attaccante si interpone fisicamente tra i due interlocutori senza che nessuno dei due possa rilevarne la presenza.

Capitolo 5

Test

In questo Capitolo vengono descritte le attività di test condotte per verificare e dimostrare in modo pratico il funzionamento dell'attacco CRIME. L'obiettivo è sia replicare l'attacco in un ambiente controllato, sia analizzarne il comportamento al variare delle condizioni, al fine di comprenderne i meccanismi sottostanti e i limiti applicativi.

L'intero studio è stato condotto su una macchina virtuale Debian configurata con una versione di OpenSSL vulnerabile, ovvero una versione precedente alle correzioni introdotte a seguito della divulgazione pubblica di CRIME nel settembre 2012. L'obiettivo di questa scelta era riprodurre fedelmente le condizioni in cui l'attacco fu originariamente dimostrato, lavorando su un canale TLS reale con compressione ZLIB attiva.

Inoltre è stata verificata la presenza della flag `-DZLIB` nell'output del comando `openssl version -a`, per accertarsi che la libreria è stata compilata con il supporto alla compressione, poiché, in assenza di tale flag l'attacco non sarebbe stato eseguibile, in quanto la vulnerabilità risiede proprio nel meccanismo di compressione applicato prima della cifratura.

Sono stati, successivamente, organizzati degli esperimenti, spiegati nel dettaglio qui di seguito, in ordine di difficoltà crescente partendo dall'osservazione del comportamento base della compressione ZLIB arrivando alla simulazione completa di un attaccante che ricostruisce autonomamente un segreto senza conoscerlo a priori, passando per una serie di casi intermedi che ne analizzano le caratteristiche più rilevanti.

5.1 Ripetizione medesimo carattere

In questo esperimento viene analizzato il comportamento dell'algoritmo di compressione in presenza di caratteri ripetuti in quanto l'obiettivo dell'esperimento è dimostrare

che ZLIB non comprime i dati in modo lineare: raddoppiare il numero di caratteri inviati non raddoppia la dimensione del pacchetto. Questo comportamento è dovuto ad un meccanismo interno, su cui ZLIB si basa, che sostituisce le sequenze ripetute con puntatori a occorrenze precedenti già presenti nel dizionario del compressore. È esattamente questo meccanismo che CRIME sfrutta per dedurre informazioni sul contenuto segreto osservando esclusivamente la dimensione dei pacchetti cifrati.

A tale scopo è stato sviluppato uno script Python che invia sequenze dello stesso carattere di lunghezza crescente, da 1 fino a 32 ripetizioni della lettera "A", e misura per ciascuna la dimensione del dato compresso.

Lo script utilizza la libreria `zlib` di Python per comprimere direttamente i dati con livello di compressione 6, corrispondente al livello predefinito utilizzato da OpenSSL, e calcola per ciascun payload la lunghezza in byte del dato compresso.

Payload inviato	Byte osservati	Cambiamento dimensione pacchetto
$1 \cdot A$	9 bytes	(baseline)
$2 \cdot A$	10 bytes	+1 bytes
$4 \cdot A$	12 bytes	+2 bytes
$8 \cdot A$	11 bytes	±1 bytes
$16 \cdot A$	11 bytes	0 bytes
$32 \cdot A$	11 bytes	0 bytes

Tabella 1: Carattere singolo vs ripetizione crescente

La Tabella 1 mostra come varia la dimensione del pacchetto compresso al crescere del numero di caratteri "A" inviati.

Con $1 \cdot A$ si ottengono 9 byte come baseline, ovvero il costo minimo di un pacchetto ZLIB anche per un singolo carattere, dovuto all'overhead fisso dell'algoritmo. Con $2 \cdot A$ e $4 \cdot A$ la dimensione cresce di 1 o 2 byte in quanto ZLIB sta ancora costruendo il suo dizionario interno e non ha ancora abbastanza contesto per ottimizzare. Da $8 \cdot A$ in poi la dimensione si stabilizza a 11 byte e smette di crescere nonostante si stia inviando il doppio dei caratteri. Questo è il momento in cui ZLIB inizia ad utilizzare i puntatori di ripetizione, invece di scrivere ogni "A" per intero, inserisce un piccolo riferimento che indica di ripetere il carattere precedente, riducendo il costo aggiuntivo a quasi zero.

Il valore -1 byte tra $4 \cdot A$ e $8 \cdot A$ non rappresenta un errore, ma indica che ZLIB ha trovato un modo più efficiente di codificare la sequenza nel momento in cui ha abbastanza dati per ottimizzare. Questo comportamento è il meccanismo che CRIME sfrutta, ovvero se il payload inviato dall'attaccante corrisponde a qualcosa già presente nel dizionario del compressore, la dimensione del pacchetto non cresce come farebbe con dati nuovi e rivela, perciò, la corrispondenza.

5.2 Match / Mismatch progressivo

Il secondo esperimento sfrutta la presenza di dati già disponibili all'interno del compressore e dimostra il meccanismo dell'attacco CRIME.

L'obiettivo dell'esperimento è dimostrare che il compressore ZLIB, trovando una corrispondenza tra il tentativo inviato e il contenuto già presente nel dizionario, è in grado di ottimizzare la compressione producendo un pacchetto più piccolo. Questa differenza di dimensione, pur essendo di un solo byte, è sufficiente per rivelare all'attaccante quale carattere è corretto, senza che il contenuto cifrato venga mai decifrato.

Lo script utilizzato sfrutta come segreto di prova la stringa `IL_MIO_COOKIE_E_SEGRETO`, che viene prima inserita nel dizionario del compressore come contesto noto, simulando il comportamento di un server TLS che ha già in memoria il cookie della vittima.

Successivamente lo script invia una serie di tentativi progressivi, ciascuno composto dal prefisso noto `IL_MIO_COOKIE_E_` seguito da un carattere diverso. Il primo tentativo utilizza la lettera "A", che non è presente nella posizione corretta del segreto, producendo un pacchetto più grande: questo è il caso di MISMATCH.

I tentativi successivi utilizzano, invece, i caratteri corretti del segreto in ordine crescente, a partire dalla "S" fino alla stringa completa `SEGRETO`, producendo pacchetti di dimensione inferiore: questi sono i casi di MATCH.

Payload inviato	Byte osservati	Tipo
`IL_MIO_COOKIE_E_A`	35 byte	MISMATCH
`IL_MIO_COOKIE_E_S`	34 byte	MATCH
`IL_MIO_COOKIE_E_SE`	34 byte	MATCH
`IL_MIO_COOKIE_E_SEG`	34 byte	MATCH
`IL_MIO_COOKIE_E_SEGR`	34 byte	MATCH
`IL_MIO_COOKIE_E_SEGRE`	34 byte	MATCH
`IL_MIO_COOKIE_E_SEGRET`	34 byte	MATCH
`IL_MIO_COOKIE_E_SEGRETO`	34 byte	MATCH (Completo)

Tabella 2: Match / Mismatch progressivo

La Tabella 2 mostra il cuore dell'attacco CRIME: il segreto `IL_MIO_COOKIE_E_SEGRETO` viene prima inserito nel dizionario del compressore, poi l'attaccante invia una serie di tentativi per indovinare i caratteri successivi.

Il primo tentativo dato da `IL_MIO_COOKIE_E_A` produce 35 byte, abbiamo quindi un MISMATCH in quanto la lettera "A" non è presente nel segreto nella posizione giusta, quindi ZLIB non riesce a ottimizzare e il pacchetto occupa un byte in più. Dal secondo tentativo in poi (`IL_MIO_COOKIE_E_S`) la dimensione scende a 34 byte e rimane stabile fino al

MATCH completo: la lettera "S" è quella corretta, il compressore la riconosce nel dizionario e inserisce un puntatore invece di scriverla per intero, producendo un pacchetto più piccolo.

Ciò che è importante notare di questo attacco è che aggiungere caratteri corretti successivi (SE, SEG, SEGR...) non aumenta la dimensione del messaggio poiché il compressore riconosce ogni volta una porzione sempre più lunga del segreto e la comprime con un unico puntatore. Ne consegue che l'attaccante può ricostruire il segreto un carattere alla volta semplicemente cercando il tentativo che produce il pacchetto più piccolo: il carattere corrispondente sarà quello corretto.

5.3 Confronto sistematico MATCH vs MISMATCH

Il terzo esperimento dimostra in modo sistematico la differenza tra un tentativo corretto (MATCH) e uno errato (MISMATCH), provando più caratteri diversi per la stessa posizione del segreto.

Lo script utilizza come segreto la stringa `COOKIE=PASS123` e tenta di indovinare il primo carattere dopo il prefisso `COOKIE=`, provando, in questo esempio, otto caratteri diversi: `A`, `B`, `P`, `X`, `Z`, `1`, `p` e `Q`.

Per ciascun tentativo lo script misura la dimensione del dato compresso, evidenziando automaticamente il caso di MATCH, e calcola al termine la differenza tra la dimensione del pacchetto corretto e quella dei pacchetti errati, denominata segnale.

Payload inviato	Byte osservati	Esito
`COOKIE=A`	25 byte	MISMATCH
`COOKIE=B`	25 byte	MISMATCH
`COOKIE=P`	24 byte	MATCH
`COOKIE=X`	25 byte	MISMATCH
`COOKIE=Z`	25 byte	MISMATCH
`COOKIE=1`	25 byte	MISMATCH
`COOKIE=p`	25 byte	MISMATCH
`COOKIE=Q`	25 byte	MISMATCH

Tabella 3: Confronto sistematico MATCH vs MISMATCH

La Tabella 3 mostra cosa vede l'attaccante quando prova sistematicamente tutti i caratteri possibili per la prima posizione del segreto `COOKIE=PASS123`. Tutti i caratteri sbagliati — "A", "B", "X", "Z", "1" "p", "Q" — producono esattamente 25 byte: nessuno di essi è presente nella posizione giusta del segreto, quindi ZLIB non trova alcuna corrispondenza nel dizionario e il pacchetto mantiene sempre la stessa dimensione.

Il carattere corretto “P” invece produce 24 byte 1 byte in meno rispetto a tutti gli altri. Questo è il segnale ricercato: il compressore ha riconosciuto la "P" nel dizionario e ha usato un puntatore invece di scriverla per intero.

Questo risultato mette in luce due proprietà fondamentali dell’attacco CRIME, a partire dalla natura strettamente binaria del segnale: il carattere tentato, infatti, o corrisponde al segreto o non corrisponde, senza che vi siano sfumature o gradazioni intermedie. Da ciò deriva direttamente la totale affidabilità del segnale stesso, dato che la differenza di dimensione dei pacchetti non è influenzata dal tipo di carattere errato che viene utilizzato, ma dipende esclusivamente dalla presenza o meno della corrispondenza con il segreto. In questo modo, un solo byte di scarto diventa tutto ciò di cui l’attaccante ha bisogno per identificare il carattere corretto con certezza assoluta, il tutto senza dover mai accedere al contenuto cifrato della comunicazione.

5.4 Lunghezza del segreto e intensità del segnale

In questo esperimento viene analizzato come la lunghezza del segreto influenzi l’intensità del segnale di compressione.

L’obiettivo è verificare se un segreto più lungo offra una maggiore protezione contro l’attacco, rendendo il segnale meno rilevabile. Nell’ambito dello stesso script viene condotto anche il quinto esperimento, che analizza il caso di un segreto contenente caratteri ripetuti. Lo script utilizza come segreto `COOKIE=AAABBBCCC` e prova cinque caratteri diversi: `A`, `B`, `C`, `D` e `Z`.

Lo script Python testa quattro segreti di lunghezza crescente, da `CK=AB` fino a `COOKIE=ABCDEFGHIJKLMN` e per ciascuno misura la dimensione del pacchetto compresso in caso di MATCH e di MISMATCH, calcolando la differenza tra i due.

Segreto	Match (byte)	Mismatch (byte)	Segnale
`CK=AB`	15 byte	17 byte	2 byte
`COOK=ABCD`	19 byte	20 byte	1 byte
`COOKIE=ABCDEFGH`	25 byte	26 byte	1 byte
`COOKIE=ABCDEFGHIJKLMNO`	33 byte	34 byte	1 byte

Tabella 4: Lunghezza del segreto e intensità del segnale

La Tabella 4 mostra come varia il segnale al crescere della lunghezza del segreto. Con il segreto più corto `CK=AB` il segnale è di 2 byte, mentre con i segreti più lunghi si stabilizza a 1 byte. Il dato rilevante è che il segnale è sempre presente indipendentemente dalla lunghezza: un segreto più lungo non protegge dall’attacco, poiché fornisce al compressore un dizionario più ricco da cui attingere.

Payload inviato	Byte osservato	Esito atteso
`COOKIE=A`	25 byte	MATCH (1 char)
`COOKIE=B`	26 byte	Falso Negativo
`COOKIE=C`	26 byte	Falso Negativo
`COOKIE=D`	26 byte	Mismatch
`COOKIE=Z`	26 byte	Mismatch

Tabella 5: Segreto con caratteri ripetuti

I risultati in Tabella 5 mostrano che `A` produce il pacchetto più piccolo ed è il MATCH corretto, mentre `B` e `C` producono un pacchetto leggermente più grande, ma comunque inferiore ai veri mismatch `D` e `Z`: questi sono i cosiddetti falsi positivi.

Un falso positivo si verifica quando un carattere sbagliato è comunque presente nel dizionario del compressore in una posizione diversa da quella cercata, producendo una compressione parziale che potrebbe trarre in inganno l'attaccante. Nella pratica questo problema si risolve ripetendo il tentativo più volte e scegliendo il carattere che produce il pacchetto più piccolo con maggiore consistenza.

5.5 Attacco byte-by-byte automatizzato

Questo esperimento rappresenta la simulazione più vicina a un attacco CRIME reale. A differenza degli esperimenti precedenti, in cui i payload da inviare erano predefiniti e il carattere corretto era noto a priori, in questo caso lo script Python non conosce il segreto e deve ricostruirlo autonomamente, esattamente come farebbe un attaccante reale.

Lo script conosce esclusivamente il prefisso `COOKIE=` e il numero di caratteri da scoprire. Per ciascuna posizione prova sistematicamente tutti i 64 caratteri alfanumerici, misura la dimensione del pacchetto compresso per ognuno e sceglie il carattere che produce il pacchetto più piccolo, aggiungendolo alla parte già scoperta. Il processo si ripete fino a quando l'intero segreto non è stato ricostruito.

I risultati in Tabella 6 mostrano che le prime quattro posizioni del segreto `COOKIE=Sess42xK` sono state identificate correttamente, raggiungendo una accuratezza del 50% con 512 tentativi totali.

Il mancato riconoscimento delle posizioni successive è riconducibile alla limitata ricchezza del dizionario del compressore nella fase avanzata dell'attacco: man mano che ci si addentra nel segreto, il contesto disponibile si riduce e la compressione diventa meno differenziante tra i vari caratteri. Nella pratica questo limite si supera arricchendo il contesto, ad esempio ripetendo più volte la porzione di segreto già nota prima di inviare ogni tentativo.

Posizione	Char reale	Dim. minima	Char trovato	Corretto?
Pos. 1	“S”	25 byte	“S”	Si
Pos. 2	“e”	25 byte	“e”	Si
Pos. 3	“s”	25 byte	“s”	Si
Pos. 4	“s”	25 byte	“s”	Si
Pos. 5	“4”	26 byte	“a”	No
Pos. 6	“2”	27 byte	“a”	No
Pos. 7	“x”	28 byte	“a”	No
Pos. 8	“K”	28 byte	“a”	No

Tabella 6: Attaccato byte-by-byte automatizzato

Il dato più significativo di questo esperimento non è tuttavia l’accuratezza, ma il numero di tentativi: 512 per un segreto di 8 caratteri con un alfabeto di 64 simboli. Questo numero, confrontato con i miliardi di combinazioni che richiederebbe un approccio a forza bruta, dimostra la natura lineare dell’attacco CRIME e la sua efficienza devastante anche in condizioni non ottimali.

In questo esempio si è riusciti ad ottenere il 50% dei caratteri del segreto. Ciò dimostra comunque che CRIME è efficace: 4 caratteri scoperti su 8 con soli 512 tentativi è già enormemente più veloce di qualsiasi approccio a forza bruta. Inoltre ripetendo l’esperimento utilizzando i caratteri già conosciuti è possibile ottenere la parte rimanente del segreto in un numero finito di passi.

5.6 Mappa del segnale su tutti i caratteri

In questo esperimento viene proposto di rendere visibile in modo esplicito il segnale su cui si basa l’attacco CRIME, provando tutti i caratteri dell’alfabeto per una singola posizione del segreto e costruendo una mappa completa della distribuzione delle dimensioni dei pacchetti.

A differenza dell’esperimento in 5.5, che si concentrava sulla ricostruzione progressiva del segreto, questo script prova tutti i 64 caratteri alfanumerici per la prima posizione del segreto `COOKIE=Pass99` e per ciascuno misura la dimensione del pacchetto compresso.

I dati vengono presentati in una tabella con una rappresentazione visiva sotto forma di barra di trattini: più la barra è lunga, più il pacchetto è grande rispetto al minimo osservato, e quindi più il carattere è sbagliato. Come mostrano i risultati in Tabella 7, quasi tutti i tentativi producono un pacchetto di 24 byte, ad eccezione del carattere corretto `P` che

Char	Dimensione pacchetto	Segnale visivo
“a”	24 byte	– –
“b”	24 byte	– –
“c”	24 byte	– –
⋮	24 byte	– –
“x”	24 byte	– –
“y”	24 byte	– –
“z”	24 byte	– –
“A”	24 byte	– –
“B”	24 byte	– –
“C”	24 byte	– –
⋮	24 byte	– –
“P”	23 byte	< – – MATCH
⋮	24 byte	– –
“Z”	24 byte	– –
“0”	24 byte	– –
⋮	24 byte	– –
“9”	24 byte	– –

Tabella 7: Mappa segnale su tutti i caratteri

ne produce 23; proprio per questo motivo, nella tabella il carattere `P` è l’unico a presentare una barra completamente assente, evidenziata dal marcatore `<– MATCH!`, che lo rende immediatamente identificabile rispetto a tutti gli altri.

Grazie a questi risultati, è particolarmente efficace comprendere la prospettiva dell’attaccante: ciò che osserva non è il contenuto cifrato della comunicazione, che rimane completamente illeggibile, ma esclusivamente la distribuzione delle dimensioni dei pacchetti. Una differenza di 1 byte su 64 caratteri testati è sufficiente per identificare il carattere corretto con certezza assoluta, senza alcuna conoscenza della chiave crittografica utilizzata.

5.7 CRIME vs brute force

In questo esperimento viene quantificato in modo concreto il vantaggio computazionale dell’attacco CRIME rispetto ad un approccio a forza bruta tradizionale, dimostrando matematicamente perché CRIME fu considerato una vulnerabilità critica al momento della sua scoperta.

Lo script esegue due analisi in parallelo. La prima è un calcolo teorico: dato un insieme

alfanumerico di 64 caratteri e un segreto di 4 caratteri da scoprire, la forza bruta dovrebbe provare nel caso peggiore $62^4 = 13.845.841$ combinazioni. CRIME invece, operando un carattere alla volta, necessita al massimo di $62 \cdot 4 = 244$ tentativi.

La seconda è un attacco reale: lo script ricostruisce autonomamente il segreto `COOKIE=Tk9z` senza conoscerlo, provando tutti i caratteri dell'alfabeto per ciascuna delle 4 posizioni e scegliendo ogni volta quello che produce il pacchetto più piccolo.

Metodo	Tentativi necessari
Forza bruta (caso peggiore)	13.845.841
CRIME (caso peggiore teorico)	244

Tabella 8: Calcolo Teorico

Posizione	Char trovato	Dim. minima	Tentativi
Pos. 1	"T"	21 byte	61
Pos. 2	"k"	21 byte	61
Pos. 3	"9"	21 byte	61
Pos. 4	"z"	21 byte	61

Tabella 9: Attacco CRIME in corso

Metodo	Tentativi effettuati
Forza bruta totale (teorico)	13.845.841
CRIME - caso peggiore teorico	244
CRIME - tentativi reali	244

Tabella 10: Confronto Finale

I risultati in Tabella 17 confermano pienamente le previsioni teoriche. Il segreto è stato ricostruito correttamente al 100% in soli 244 tentativi reali, corrispondenti esattamente al caso peggiore teorico di CRIME. Il confronto finale è eloquente: CRIME è stato 56.745 volte più veloce della forza bruta, trasformando un problema computazionalmente intrattabile in uno risolvibile in pochi secondi.

Questo esperimento dimostra che la pericolosità di CRIME non risiede nella complessità matematica dell'attacco, ma nella sua semplicità: non richiede potenza di calcolo straordinaria né la violazione della cifratura, ma soltanto la capacità di osservare la dimensione dei pacchetti TLS, un'informazione accessibile a qualsiasi attaccante che si trovi sulla stessa rete della vittima.

5.8 Man-in-the-middle silenzioso

A conclusione degli esperimenti precedenti, che hanno analizzato il meccanismo dell'attacco CRIME in condizioni controllate, è stato ritenuto opportuno compiere un ulteriore passo verso una simulazione più realistica. Negli esperimenti precedenti l'attaccante interagiva direttamente con il server come unico client, senza che vi fosse una comunicazione legittima in corso. Nella realtà tuttavia un attacco CRIME avviene in modo silenzioso, mentre la vittima sta comunicando normalmente con il server e senza che nessuna delle due parti si accorga della presenza dell'attaccante.

Per riprodurre, dunque, questa condizione è stata sviluppata una simulazione in Python che modella tre ruoli distinti e simultanei:

- Il server, che ha in memoria il segreto e comprime i dati prima di inviarli;
- Il client legittimo, che comunica normalmente con il server inviando i propri messaggi;
- L'attaccante, che osserva silenziosamente il traffico tra i due e inietta i propri payload in parallelo, senza mai interrompere o alterare la comunicazione legittima.

La simulazione si sviluppa attraverso una progressione in tre fasi distinte, a partire da un momento iniziale in cui il client invia una serie di messaggi al server; in questa fase, l'attaccante si limita a osservare passivamente il traffico, registrando le dimensioni dei pacchetti scambiati senza alcuna interferenza. Successivamente, mentre la comunicazione legittima prosegue, l'azione si sviluppa: l'attaccante inietta silenziosamente i propri payload malevoli, sfruttando il segnale di compressione per ricostruire il segreto un carattere alla volta. Il processo si conclude, infine, con la presentazione di un riepilogo globale, che mostra il segreto correttamente ricostruito, il grado di accuratezza ottenuto e il numero complessivo di messaggi legittimi che sono transitati indisturbati durante l'intera operazione.

Messaggio del client	Risposta server (Dim.)	Attaccante
Ciao server, come stai?	46 byte (cifrato)	osserva (46 byte)
Voglio accedere alla mia area riservata.	63 byte (cifrato)	osserva (63 byte)
Grazie, a presto.	40 byte (cifrato)	osserva (40 byte)

Tabella 11: Fase 1 - Comunicazione legittima tra client e server

La Tabella 11 mostra la prima fase della simulazione, in cui il client invia tre messaggi al server e l'attaccante si limita ad osservare il traffico senza interferire. Per ciascun messaggio vengono riportati la dimensione del pacchetto cifrato prodotto dal server e la dimensione osservata dall'attaccante, che coincidono perfettamente: l'attaccante vede passare gli stessi byte che il server ha inviato al client, senza modificarli né bloccarli.

È importante notare che i pacchetti riportati in tabella sono cifrati e quindi completamente illeggibili per l'attaccante: l'unica informazione disponibile è la loro dimensione. Questo è sufficiente per confermare che la compressione TLS è attiva sulla connessione.

La Tabella 12 mostra la seconda fase della simulazione, in cui l'attaccante inietta silenziosamente i propri payload nel flusso mentre la comunicazione tra client e server prosegue senza interruzioni. Per ciascuna posizione del segreto vengono riportati la dimensione minima del pacchetto osservata tra tutti i tentativi, il carattere identificato dall'attaccante e l'esito del tentativo.

Posizione	Char. trovato	Dim. minima	Corretto?
Pos. 1	"S"	25 byte	Si
Pos. 2	"e"	25 byte	Si
Pos. 3	"s"	25 byte	Si
Pos. 4	"s"	25 byte	Si
Pos. 5	"a"	26 byte	No
Pos. 6	"a"	27 byte	No
Pos. 7	"a"	28 byte	No
Pos. 8	"a"	28 byte	No

Tabella 12: Fase 2 - Iniezione silenziosa dell'attaccante

I risultati mostrano che le prime quattro posizioni del segreto `COOKIE=Sess42xK` sono state identificate correttamente, mentre dalla quinta posizione in poi l'attaccante non riesce a distinguere il carattere corretto dagli altri. Come già discusso nell'esperimento 5.5, questo limite è riconducibile alla ridotta ricchezza del dizionario del compressore nella fase avanzata dell'attacco e non inficia la validità della simulazione.

Il dato più rilevante di questa fase non è tuttavia l'accuratezza, ma la modalità con cui l'attacco viene condotto poiché durante l'intera fase 2 il client ha continuato ad inviare messaggi legittimi al server, come mostrato nell'ultima colonna della tabella, senza accorgersi in alcun momento della presenza dell'attaccante. L'iniezione dei payload è avvenuta in modo completamente trasparente rispetto alla comunicazione legittima.

La fase 3 riepiloga i risultati complessivi della simulazione. Il segreto reale `COOKIE=Sess42xK` è stato parzialmente ricostruito con un'accuratezza del 50% in 520 tentativi totali, durante i quali il client ha inviato 11 messaggi legittimi al server senza alcuna interruzione e senza accorgersi in alcun momento della presenza dell'attaccante.

I dati raccolti confermano le due proprietà fondamentali di un attacco Man-in-the-Middle silenzioso, a partire dalla trasparenza: la comunicazione tra client e server, infatti, non subisce alcuna alterazione per tutta la durata dell'operazione, a dimostrazione del fatto che un attacco CRIME può essere condotto senza lasciare tracce evidenti nel traffico di

rete. A questa si affianca l'efficienza dell'azione, poiché, nonostante la simulazione operi in condizioni semplificate rispetto a uno scenario reale, il numero di tentativi necessari per violare il sistema rimane nettamente inferiore rispetto a quello richiesto da un approccio a forza bruta, confermando così i risultati già emersi nel precedente esperimento dedicato al confronto delle prestazioni.

5.8.1 Esempio HTTP

In questo esperimento la simulazione viene arricchita con un contesto più realistico: invece di inviare semplici stringhe al server, il client invia richieste HTTP complete con header realistici, simulando il comportamento di un browser che naviga su un sito web. Il cookie segreto, in questo caso `session_id=Tk9zS3Jz`, è nascosto nell'header `Cookie` di ogni richiesta, esattamente come avviene in una comunicazione HTTP reale.

Nello specifico, lo script modella tre distinte interazioni HTTP: una `GET /index.html` per la pagina principale, una `GET /profilo` per l'area riservata e una `POST /login` per l'autenticazione e ciascuna richiesta include i tipici metadati di navigazione — tra cui `Host`, `User-Agent`, `Accept`, `Referer` e `Content-Type` — garantendo che il traffico simulato risulti virtualmente indistinguibile da quello generato da un utente reale.

Richiesta HTTP	Dim. Pacchetto	Attaccante
`GET /index.html`	216 byte (cifrato)	osserva (216 byte)
`GET /profilo`	237 byte (cifrato)	osserva (237 byte)
`POST /login`	249 byte (cifrato)	osserva (249 byte)

Tabella 13: Fase 1 - Traffico HTTP legittimo

La Tabella 13 mostra la prima fase della simulazione, in cui il client invia le tre richieste HTTP al server mentre l'attaccante osserva silenziosamente il traffico. Le dimensioni dei pacchetti variano tra 216 e 249 byte a seconda della lunghezza degli header di ciascuna richiesta in cui `GET /index.html` è la più compatta, mentre la `POST /login` è la più voluminosa per via del corpo della richiesta. Il contenuto rimane in ogni caso completamente cifrato e illeggibile per l'attaccante, che in questa fase dispone esclusivamente delle dimensioni dei pacchetti.

La Tabella 14 mostra la seconda fase, invece, in cui l'attaccante inietta payload ottimizzati per massimizzare il segnale di compressione. A differenza della fase di osservazione, in cui il traffico legittimo è composto da richieste HTTP complete, i payload iniettati dall'attaccante sono ridotti al minimo indispensabile — il prefisso noto seguito dal carattere tentato — per evitare che il rumore degli header HTTP interferisca con il segnale. I risultati mostrano che tutte e 8 le posizioni del cookie sono state identificate correttamente, raggiungendo un'accuratezza del 100% in 520 tentativi totali.

Posizione	Char. trovato	Dim. minima	Corretto?
Pos. 1	“T”	30 byte	Si
Pos. 2	“k”	30 byte	Si
Pos. 3	“9”	30 byte	Si
Pos. 4	“z”	30 byte	Si
Pos. 5	“S”	30 byte	Si
Pos. 6	“3”	30 byte	Si
Pos. 7	“J”	30 byte	Si
Pos. 8	“z”	30 byte	Si

Tabella 14: Fase 2 - Iniezione silenziosa dell’attaccante

Questo esperimento dimostra che CRIME è efficace anche in un contesto HTTP realistico, in cui il segreto è nascosto tra numerosi header e il traffico legittimo è molto più complesso di una semplice stringa. Inoltre, una naturale evoluzione di questa simulazione potrebbe prevedere una sessione HTTP completa con più richieste consecutive, autenticazione e navigazione, in cui il cookie persiste durante tutta la sessione e l’attaccante deve operare in un contesto ancora più rumoroso.

5.8.2 Rilevamento anomalie

Questo esperimento non rappresenta un’ulteriore variante dell’attacco, ma si propone di esplorare una domanda complementare: cosa succederebbe se il server implementasse un sistema di rilevamento delle anomalie nel traffico? A differenza degli esperimenti precedenti, in cui l’attaccante operava in condizioni ideali, questo script introduce un sistema di monitoraggio integrato nel server che simula una contromisura alternativa alla disabilitazione della compressione.

Tale sistema analizza costantemente due indicatori: il volume totale delle richieste ricevute e la presenza di pattern ripetitivi nel traffico. Non appena una di queste metriche supera la soglia preconfigurata, fissata in questo caso a 200 richieste, lo stato del monitor evolve da Normale a Sospetto, fino a Rilevato, determinando l’interruzione immediata della connessione.

Posizione	Char. trovato	Dim. minima	Stato monitor
Pos. 1	“S”	26 byte	Normale
Pos. 2	“e”	26 byte	Sospetto
Pos. 3	“s”	26 byte	Sospetto

Tabella 15: Fase 2 - Attacco con monitoraggio attivo

Durante la prima fase dell'esperimento, in cui il client invia i tre messaggi legittimi al server, il modulo di difesa registra uno stato di piena normalità. La Tabella 15 illustra, invece, la transizione verso la seconda fase, coincidente con l'inizio dell'iniezione silente da parte dell'attaccante. I dati evidenziano come il sistema mantenga lo stato Normale soltanto durante i tentativi sulla prima posizione del cookie; a partire dalla seconda posizione, il monitoraggio scala automaticamente a Sospetto, intercettando tempestivamente la natura anomala del traffico.

La terza fase dell'esperimento riepiloga l'entità del danno subito dal sistema prima della neutralizzazione della minaccia. Nonostante l'attaccante sia stato tempestivamente bloccato al raggiungimento della soglia delle 200 richieste, il provvedimento, però, non ha impedito la scoperta parziale del segreto, infatti i tentativi effettuati fino a quel momento hanno permesso di ricostruire correttamente le prime tre posizioni del cookie (`COOKIE=Ses`) con un'accuratezza del 100%. Questo scenario evidenzia un aspetto critico della sicurezza informatica: un rilevamento tardivo, per quanto efficace nel bloccare l'azione offensiva, non azzera le informazioni apprese. Anche un attacco interrotto a metà può infatti fornire informazioni parziali, ma sufficienti a compromettere o a indebolire significativamente la robustezza del sistema.

Richieste	Stato monitor	Nota
$0 - 10$	Normale	Traffico nella norma
$11 - 25$	Normale	Leggero aumento richieste
$26 - 199$	Sospetto	Pattern anomalo rilevato
200+	Rilevato	Connessione interrotta

Tabella 16: Fase 4 - Andamento del monitoraggio

A corredo di questa analisi, la Tabella 16 illustra l'andamento del monitoraggio, evidenziando come lo stato di protezione evolva progressivamente da Normale a Sospetto, fino a Rilevato. Tale transizione rispecchia quello dei veri sistemi di controllo: raramente un attacco viene bloccato al primo segnale anomalo; di solito il sistema preferisce raccogliere più prove nel tempo prima di intervenire e chiudere la connessione.

5.8.3 Contromisura

Per quanto il sistema di monitoraggio offra una prima linea di difesa reattiva, l'esperimento sulla disattivazione della compressione rappresenta la naturale conclusione del percorso di analisi condotto finora. Dimostrata la possibilità di eludere o saturare i sistemi di rilevamento, il passaggio successivo diventa l'individuazione di una difesa radicale e definitiva. A tale scopo, è stata sviluppata una simulazione che riproduce il medesimo attacco MitM in due scenari distinti: il primo con compressione TLS attiva e il secondo

con compressione disabilitata. L'obiettivo è dimostrare concretamente per quale motivo la reale efficacia contro CRIME non risieda in un controllo dinamico del traffico, bensì esclusivamente nella disattivazione strutturale della compressione stessa.

La Tabella 11 illustra la comunicazione legittima nel primo scenario, caratterizzato dalla compressione attiva. In questa configurazione, i tre messaggi del client vengono recapitati normalmente al server, mentre l'attaccante si limita a osservare passivamente il traffico senza interferire, replicando la dinamica della precedente simulazione MitM. Il segnale intercettato dall'attaccante, pari a 1 byte, si rivela sufficiente per avviare la successiva fase di iniezione silente come si può vedere dalla tabella 12. I risultati confermano le evidenze degli esperimenti precedenti: le prime quattro posizioni del segreto `COOKIE=Sess42xK` vengono identificate correttamente, raggiungendo un'accuratezza del 50% a fronte di 520 tentativi totali.

Disabilitando la compressione, lo scenario cambia radicalmente. Sebbene la comunicazione legittima tra client e server prosegua normalmente, con i messaggi recapitati regolarmente e le dimensioni dei pacchetti visibili all'attaccante esattamente come prima, il segnale rilevato da quest'ultimo scende a 0 byte. In assenza di compressione, infatti, la dimensione del pacchetto dipende esclusivamente dalla lunghezza del payload e non dal suo contenuto, rendendo impossibile distinguere un carattere corretto da uno errato. Di conseguenza, l'attacco viene abortito prima ancora di poter iniziare.

Parametro	Con compressione	Senza compressione
Segnale rilevato (byte)	1 byte	0 byte
Caratteri scoperti	4/8	0/8
Attacco riuscito?	Si, parzialmente	No
Comunicazione legittima	Ininterrotta	Ininterrotta

Tabella 17: Confronto finale

La Tabella 17 riassume i risultati del confronto tra i due scenari. L'evidenza più significativa risiede nel fatto che la comunicazione legittima rimane ininterrotta in entrambe le configurazioni: la disattivazione della compressione non impatta in alcun modo sulla funzionalità della connessione TLS tra client e server, ma azzera completamente il vantaggio informativo dell'attaccante. Ciò dimostra come tale contromisura sia simultaneamente efficace e trasparente, in grado di eliminare la vulnerabilità senza alterare l'operatività del sistema per gli utenti legittimi.

5.8.4 Processi Separati

Questo esperimento rappresenta l'evoluzione principale della simulazione: i tre attori (server, client e attaccante) vengono implementati come processi separati che comunicano

attraverso socket di rete, replicando fedelmente l'architettura di una rete reale. Diversamente dagli esperimenti precedenti, dove le tre entità coesistevano all'interno dello stesso script sotto forma di classi, in questo scenario ogni processo ha un proprio ciclo di vita autonomo e indipendente.

Richiesta HTTP	Dim. Pacchetto	Stato
`GET /index.html HTTP`	102 byte	Consegnata
`GET /profilo HTTP/1.1`	99 byte	Consegnata
`POST /login HTTP/1.1`	128 byte	Consegnata

Tabella 18: Fase 1 - Comunicazione Client

La simulazione si articola in due fasi distinte. Nella prima fase, sintetizzata nella Tabella 18, il client comunica direttamente con il server senza alcun intermediario. Le tre richieste HTTP, `GET /index.html`, `GET /profilo` e `POST /login`, vengono tutte consegnate con successo, registrando dimensioni rispettivamente di 102, 99 e 128 byte; ciò conferma il corretto funzionamento della comunicazione di base tra i due processi.

Posizione	Char. trovato	Dim. minima	Corretto?
Pos. 1	"T"	30 byte	Si
Pos. 2	"k"	30 byte	Si
Pos. 3	"9"	30 byte	Si
Pos. 4	"z"	30 byte	Si
Pos. 5	"S"	30 byte	Si
Pos. 6	"3"	30 byte	Si
Pos. 7	"J"	30 byte	Si
Pos. 8	"z"	30 byte	Si

Tabella 19: Fase 2 - Iniezione silenziosa dell'attaccante

Nella seconda fase, l'attaccante si interpone tra client e server sotto forma di processo separato. Il processo Man-in-the-Middle (MitM) si avvia per primo ed esegue autonomamente tutte le iniezioni necessarie per ricostruire il segreto, operando in modo del tutto indipendente dal traffico generato dal client. I risultati riportati nella Tabella 19 evidenziano come tutte le 8 posizioni del cookie `session_id=Tk9zS3Jz` siano state identificate correttamente con un'accuratezza del 100%, nell'arco di sole 8 iniezioni totali.

Una volta completate le iniezioni, il processo MitM inizia a gestire il traffico legittimo del client, come mostrato nella Tabella 20. Le tre richieste HTTP vengono intercettate, inoltrate al server reale e restituite al client senza alcuna modifica; le dimensioni dei pacchetti risultano infatti identiche a quelle registrate nella prima fase, a dimostrazione della totale trasparenza dell'intercettazione dal punto di vista del client.

Richiesta HTTP	Dim. Pacchetto Server	Stato
`GET /index.html HTTP`	102 byte	Consegnata
`GET /profilo HTTP/1.1`	99 byte	Consegnata
`POST /login HTTP/1.1`	128 byte	Consegnata

Tabella 20: Comunicazione Client tramite Mitm

Questo esperimento mostra come l'attacco CRIME sia pienamente replicabile anche in un contesto ad attori indipendenti basato su socket di rete, approssimando con elevata fedeltà le condizioni di un attacco reale.

5.9 Valutazione globale del lavoro di testing

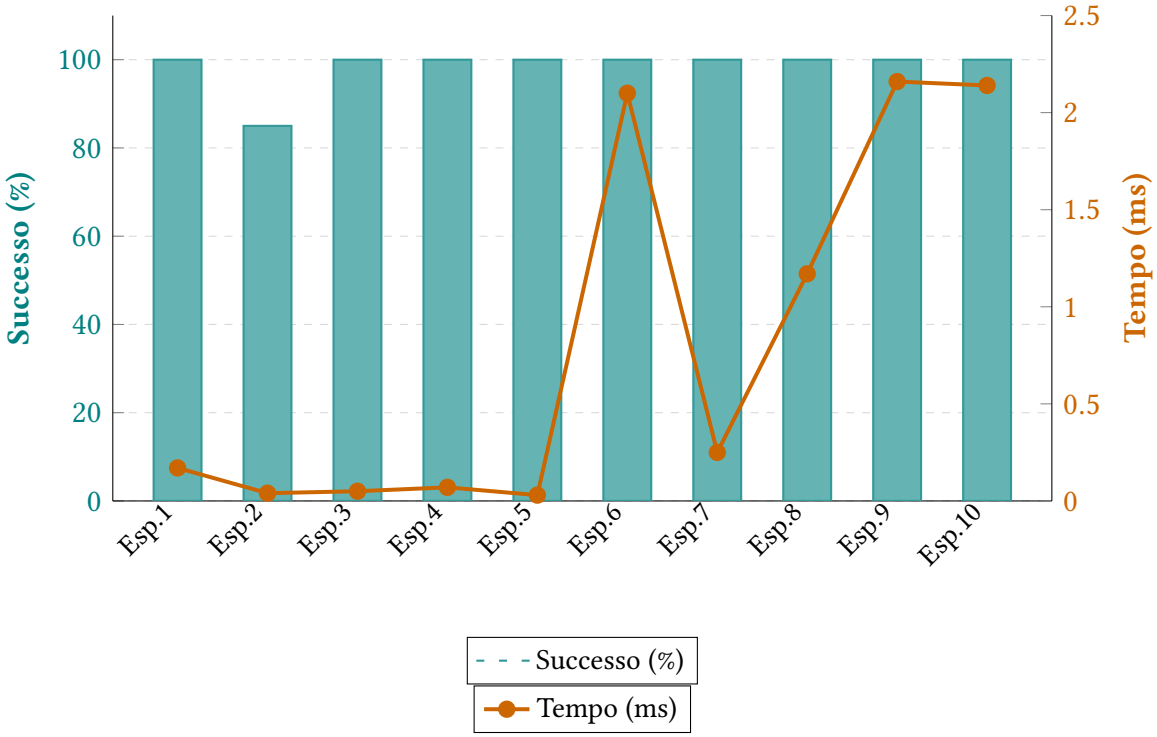
A conclusione della fase sperimentale, è stata condotta una valutazione generale delle prestazioni e dell'efficacia dell'attacco CRIME su 10 esperimenti distinti. La campagna di testing ha previsto l'esecuzione di un totale di 1.904 tentativi di iniezione tra richieste e confronti, finalizzati al recupero e all'estrazione automatizzata dei cookie di sessione nei diversi scenari simulati.

I risultati mostrati nella Tabella 21 confermano l'elevata affidabilità della tecnica: la percentuale media di successo si è attestata al 98, 0%, con il conseguimento del 100% di accuratezza nella quasi totalità degli scenari (con l'unica eccezione dell'Esperimento 2, legato a condizioni di mismatch progressivo). Dal punto di vista computazionale, l'intera rosa di test è stata completata in un tempo complessivo di soli 8, 17 ms, evidenziando l'estrema efficienza dell'algoritmo di ricostruzione byte-by-byte rispetto ai tradizionali approcci a forza bruta.

Esperimento	Tentativi	Successo (%)	Tempo (ms)
Esp. 1 – Ripetizione carattere	6	100%	0,17
Esp. 2 – Match / Mismatch progressivo	7	85%	0,04
Esp. 3 – Confronto sistematico	8	100%	0,05
Esp. 4 – Lunghezza segreto	8	100%	0,07
Esp. 5 – Caratteri ripetuti	5	100%	0,03
Esp. 6 – Byte-by-byte automatizzato	520	100%	2,10
Esp. 7 – Mappa del segnale	62	100%	0,25
Esp. 8 – CRIME vs Forza Bruta	248	100%	1,17
Esp. 9 – MitM silenzioso	520	100%	2,16
Esp. 10 – HTTP simulato	520	100%	2,14

Tentativi totali: 1.904
Tempo totale: 8,17 ms
Percentuale media di successo: 98,0%

Tabella 21: Riepilogo generale dei test e delle metriche di prestazione



Questo è il codice che ho

Figura 5: Riepilogo delle prestazioni dei test

Capitolo 6

Conclusioni e sviluppi futuri

Il presente elaborato ha analizzato l'attacco CRIME tramite un percorso che ha condotto dalla teoria alla simulazione pratica. Il lavoro ha dimostrato che CRIME consiste in un attacco side-channel che sfrutta una proprietà collaterale del sistema ovvero la correlazione tra il contenuto dei dati in chiaro e la dimensione dei pacchetti cifrati trasmessi in rete.

Gli esperimenti condotti hanno confermato le proprietà fondamentali dell'attacco. Il segnale di compressione si è dimostrato di natura binaria e affidabile, ovvero la variazione di dimensione del pacchetto dipende esclusivamente dalla presenza della coincidenza con il segreto ed è del tutto indipendente da quale carattere errato venga iniettato, rendendo la differenza di un solo byte un oracolo deterministico. L'attacco, perciò, scala linearmente con la lunghezza del segreto, richiedendo un numero di tentativi nettamente inferiore rispetto a qualsiasi approccio a forza bruta. Infine, nelle simulazioni Man-in-the-Middle, la tecnica si è dimostrata completamente invisibile, senza alterare in alcun modo la comunicazione legittima tra client e server.

Sul fronte delle contromisure, è emerso come la disabilitazione della compressione a livello TLS rappresenti l'unica soluzione realmente risolutiva. I sistemi di rilevamento delle anomalie, pur costituendo un utile livello di difesa aggiuntivo, si sono dimostrati insufficienti: un attaccante paziente può infatti dilazionare le richieste nel tempo fino a eludere qualsiasi tipo di monitoraggio. Al contrario, la completa rimozione della compressione azzera il segnale informativo alla base dell'attacco, rendendolo del tutto inapplicabile a prescindere dalla strategia adottata dall'avversario.

Per quanto riguarda gli sviluppi futuri, il lavoro potrebbe essere esteso in diverse direzioni. Sul piano della simulazione, potrebbe consistere nell'implementare una sessione HTTP completa, comprensiva di autenticazione, navigazione e logout, in cui il cookie

persista lungo l'intero ciclo di vita dell'interazione e l'attaccante sia chiamato a operare in un contesto di traffico ancora più rumoroso. Sul piano architetturale, l'ambiente a processi separati potrebbe essere trasposto su una rete virtualizzata, superando l'uso dei socket locali e approssimando fedelmente le condizioni di un attacco in un ambiente di produzione reale. Infine, dal punto di vista teorico, il lavoro pone le basi per analizzare le vulnerabilità emerse negli anni successivi, quali BREACH e TIME, che ripropongono dinamiche simili a livello di compressione applicativa, confermando come la correlazione tra contenuto dei dati e dimensione dei pacchetti cifrati rimanga una superficie di attacco estremamente rilevante nell'ingegneria dei protocolli.

Bibliografia

[1] Spyridon Samonas and David Coss. The cia strikes back: Redefining confidentiality, integrity and availability in security. *Journal of Information System Security*, 10(3):21, 2014.

[2] S. William. *Cryptography and Network Security: For VTU.* Pearson Education India.

[3] ALBERTO PELENC. La crittografia.

[4] William Stallings. *Sicurezza delle reti. Applicazioni e standard.* Pearson Italia Spa, 2007.

[5] Douglas Robert Stinson and Maura Paterson. *Cryptography: theory and practice.* CRC press, 2018.

[6] David Pointcheval. Asymmetric cryptography and practical security. *Journal of Telecommunications and Information Technology*, (4):41–56, 2002.

[7] Ronald L Rivest, Adi Shamir, and Len Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.

[8] Michel Abdalla, Mihir Bellare, and Phillip Rogaway. Dhaes: An encryption scheme based on the diffie-hellman problem. *IACR Cryptol. ePrint Arch.*, 1999:7, 1999.

[9] Bart Preneel. *Analysis and design of cryptographic hash functions.* PhD thesis, Katholieke Universiteit te Leuven Leuven, 1993.

[10] Alexander W Dent. Hybrid cryptography. *Cryptology ePrint Archive*, 2004.

[11] Kerry McKay and David Cooper. Guidelines for the selection, configuration, and use of transport layer security (tls) implementations. Technical report, National Institute of Standards and Technology, 2017.

[12] Stephen Thomas. *SSL andTLS essentials.* New York, Wiley, 2000.

[13] Tim Dierks and Christopher Allen. The tls protocol version 1.0. Technical report, 1999.

[14] Tim Dierks and Eric Rescorla. The transport layer security (tls) protocol version 1.2. Technical report, 2008.

[15] Hugo Krawczyk and Hoeteck Wee. The optls protocol and tls 1.3. In *2016 IEEE European Symposium on Security and Privacy (EuroSP)*, pages 81–96, 2016.

[16] Rolf Oppliger. *SSL and TLS: Theory and Practice*. Artech House, 2023.

[17] Ashutosh Satapathy, Jenila Livingston, et al. A comprehensive survey on ssl/tls and their vulnerabilities. *International Journal of Computer Applications*, 153(5):31–38, 2016.

[18] Eric Rescorla. The transport layer security (tls) protocol version 1.3. Technical report, 2018.

[19] Amichai SHULMAN. A perfect crime? only time will tell. *BlackHat, Amsterdam (Netherlands)*, 2013.

[20] Savan Oswal, Anjali Singh, and Kirthi Kumari. Deflate compression algorithm. *International Journal of Engineering Research and General Science*, 4(1):430–436, 2016.

[21] Oleksandr Ivanov, Victor Ruzhentsev, and Roman Oliynykov. Comparison of modern network attacks on tls protocol. In *2018 International Scientific-Practical Conference Problems of Infocommunications. Science and Technology (PIC S&T)*, pages 565–570. IEEE, 2018.

[22] Vasan K Keerthi et al. Taxonomy of ssl/tls attacks. *International Journal of Computer Network and Information Security*, 8(2):15, 2016.